



Python Programming





SpiralTrain
developer courses

General Course Remarks

- **Course times**
- **Drinks**
- **Meals**
- **Toilettes**
- **Smoking**
- **Mobile Phones**
- **Manual**
- **Software**

Everything should be made
as simple as possible,
but not simpler

Albert Einstein

Ja, was ich hier geschrieben habe, macht im Einzelnen überhaupt nicht den Anspruch auf Neuheit; und darum gebe ich auch keine Quellen an, weil es mir gleichgültig ist, ob das was ich gedacht habe, vor mir schon ein anderer gedacht hat.
Ludwig Wittgenstein

© copyright : spiraltrain@gmail.com

Course Schedule

- **Python Intro**
- Variables and Data Types
- Data Structures
- Control Flow and Operators
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes
- Optional : Python Libraries

- **What is Python?**
- **Python Features**
- **History of Python**
- **Getting Started**
- **Setting up PATH**
- **Running Python**
- **Python in Interactive Mode**
- **Python in Script Mode**
- **Python Identifiers**
- **Python Reserved Words**
- **Comments in Python**
- **Lines and Indentation**
- **Multi Line Statements**
- **Quotes in Python**

© copyright : spiraltrain@gmail.com

What is Python?

- **Python is a general purpose high-level scripting language :**
  - Supports many applications from text processing to WWW browsers and games
- **Python is Beginner's Language :**
  - Great language for the beginner programmers
- **Python is interpreted :**
  - Processed at runtime by the interpreter like Perl and PHP code
  - No need to compile your program before executing it
- **Python is interactive :**
  - Prompt allows you to interact with the interpreter directly to write your programs
- **Python is Object Oriented :**
  - Supports Object-Oriented style programming that encapsulates code within objects
- **Python is easy to learn and easy to read :**
  - Python has relatively few keywords, simple structure, and a clearly defined syntax
  - Python code is much more clearly defined and visible to the eye

Python Features

- **Broad standard library :**
  - Bulk of the library is cross portable on UNIX, Windows, and Macintosh
- **Portable :**
  - Python runs on a wide variety of platforms and has same interface on all platforms
- **Extendable :**
  - Python allows the addition of low-level modules to the Python interpreter
- **Databases :**
  - Python provides interfaces to all major commercial databases
- **GUI Programming :**
  - Python supports GUI applications that can be ported to many system calls
- **Web Applications :**
  - Django and Flask Framework allow building of Python Web Applications
- **Scalable :**
  - Python provides better structure and support for large programs than shell scripting
- **Supports automatic garbage collection :**
  - Can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java

History of Python

- **Python was developed by Guido van Rossum around 1990 at :**
  - National Research Institute for Mathematics and Computer Science in Netherlands
- **Python is derived from many other languages :**
  - Including Modula-3, C, C++,SmallTalk, Unix shell and other scripting languages
- **Python is usable as :**
  - Scripting language or compiled to byte-code for building large applications
- **Python is copyrighted and Python source code available :**
  - Like Perl under the GNU General Public License (GPL)
- **Python is maintained by a development team at the institute :**
  - Guido van Rossum still holds a vital role in directing it's progress
- **Python supports :**
  - Functional and structured programming methods as well as OOP
  - Very high-level dynamic data types and dynamic type checking

Getting Started

- **Python Official Website :**
  - `http://www.python.org/`
  - Has most up-to-date and current source code and binaries
  - Both Python 2 and Python 3 are available which differ significantly
- **Linux Installation :**
  - Download the zipped source code available for Unix/Linux
  - Extract files and enter `make install`
  - This will install Python in a standard location `/usr/local/bin`
- **Windows Installation :**
  - Download the Windows installer `python-XYZ.msi` file
  - Double-click the `msi` file and run the Python installation wizard
  - Accept the default settings or choose an installation directory
- **Python Documentation Website :**
  - `www.python.org/doc/`
  - Documentation available in HTML, PDF and PostScript format

Setting up PATH

- **To invoke the Python interpreter from any particular directory :**
  - Add the Python directory to your path
- **PATH is an environment variable :**
  - Provides a search path that lists directories that OS searches for executables
  - Named PATH in Unix or Path in Windows
  - Unix is case-sensitive, Windows is not
- **Setting PATH in Linux :**
  - In the bash shell type :

    `export PATH="$PATH:/usr/local/bin/python"` and press Enter
- **Setting PATH in Windows :**
  - At a command prompt type :

    `PATH=%PATH%;C:\Python` and press Enter
  - Or use the environment variables tab under System Properties
- **PYTHONPATH :**
  - Tells Python interpreter where to locate module files you import into a program

Running Python

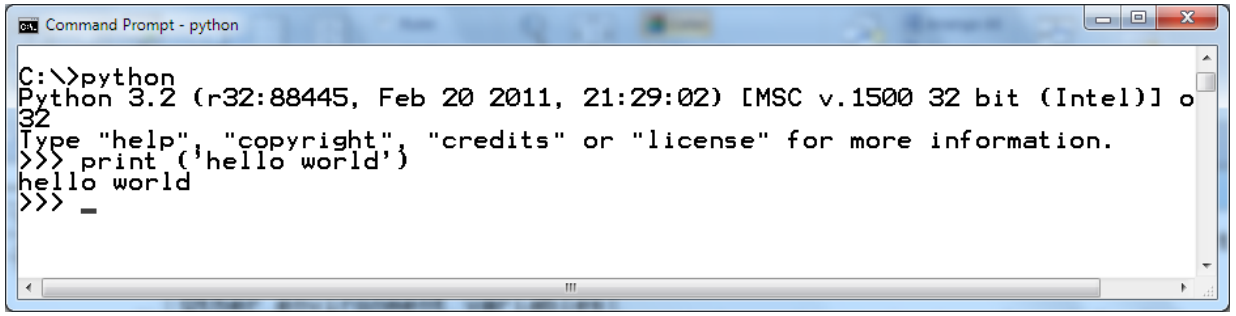
- **Invoke Python interpreter from any particular directory :**
  - Add the Python directory to your path
- **Interactive Interpreter :**
  - Start the Python interactive interpreter from command line by entering `python` :
  - You will get `$python` in Linux and `>>>` in Windows or Anaconda prompt with `IPython`
  - Script can be parameterized with command line arguments `C:\ python -h` :
- **Script from Command Line :**
  - A Python script can be executed at command line by invoking the interpreter

  `$ python script.py` in Linux

  `C:> python script.py` in Windows
- **Integrated Development Environment :**
  - Run Python from a graphical user interface (GUI) environment
  - GUI IDE for Python on system that supports Python is IDLE
- **Jupyter Notebook on a web server :**
  - Give command `jupyter notebook` from Anaconda prompt

Python in Interactive Mode

- **Python language has many similarities to Perl, C, and Java :**
  - There are some definite differences between the languages
- **Start with Hello World Python program in interactive mode :**
  - Invoking interpreter by typing `python` in a command prompt :



- **Type following right of the `python` prompt and press Enter :**

```
>>> print ("Hello, Python!")
```

- **This will produce following result :**

```
Hello, Python!
```

- **Separate multiple statements with a semicolon ;**

Python in Script Mode

- **When the interpreter is invoked with a script parameter :**
  - The script begins execution and continues until it is finished
  - When the script is finished, the interpreter is no longer active
- **All Python files will have extension `.py` :**
  - Put the following source code in a `test.py` file

```
print ("Hello, Python!")
```

- **Run the script as follows :**

```
C:\ python test.py
```

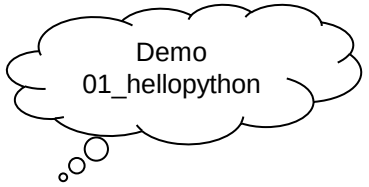
- **In a Unix environment make file executable with :**

```
$ chmod +x test.py     # set execute permission
```

- **This will produce following result :**

```
Hello, Python!
```

- `print()` **default prints a new line**



Python Identifiers

- **Names used to identify things in Python :**
  - Variable, function, class, module, or other object
- **Identifier Rules :**
  - Starts with a letter A to Z or a to z or an underscore (_)
  - Followed by zero or more letters, underscores, and digits (0 to 9)
  - Punctuation characters such as @, $, and % not allowed within identifiers
- **Python is a case sensitive programming language :**
  - `myVar` and `Myvar` are different
- **Naming conventions for Python :**
  - Class names start with uppercase letters, all other identifiers with lowercase letters
  - Leading underscore is a weak private indicator :
    - `from M import *` does not import objects with name starting with an underscore
  - Two leading underscores indicates a strongly private identifier :
    - Name mangling is applied to such identifiers by putting class name in front
  - If identifier also ends with two trailing underscores :
    - identifier is a language-defined special name

Python Reserved Words

- **May not be used as constant or variable or other identifier name**

```
and         exec        not
assert      finally     or
break       for         pass
class       from        print
continue    global      raise
def         if          return
del         import      try
elif        in          while
else        is          with
except      lambda      yield
```

Comments in Python

- **Comments starts with :**
  - A hash sign (#) that is not inside a string literal
  - All characters after the # and up to the physical line end are part of the comment

```
# First comment
print ("Hello, Python!")  # second comment
```

- **Will produce following result :**

```
Hello, Python!
```

- **Comment may be on same line after statement or expression :**

```
name = "Einstein" # This is again comment
```

- **Comment multiple lines as follows :**

```
# This is a comment.
# This is a comment, too.
# This is a comment, too.
# I said that already.
```

Lines and Indentation

- **Blocks of code are denoted by line indentation :**
  - No braces for blocks of code for class and function definitions or flow control
- **Number of spaces in the indentation is variable :**
  - All statements within the block must be indented the same amount

```
if True:            # Both blocks in first example are fine
    print ("True")
else:
  print ("False")
```

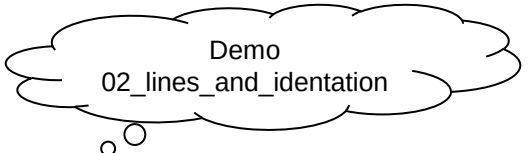
- **Other example :**

```
if True:            # Second line in second block will generate an error
    print ("Answer")
    print ("True")
else:
    print ("Answer")
  print ("False")
```

- `print()` **without new line is written as follows :**

```
print ('hello python', end='')
```

- **Semicolon ; allows multiple statements on single line**



Multi Line Statements

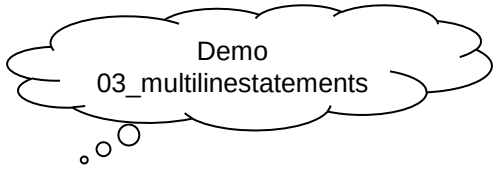
- **Statements in Python typically end with a newline**
- **Python allows the use of the line continuation character `(\)` :**
  - Denotes that the line should continue

```
total = item_one + \
        item_two + \
        item_three
```

- **Statements contained within the `[]`, `{}`, or `()` brackets :**
  - No need to use the line continuation character

```
days = ['Monday', 'Tuesday', 'Wednesday',
             'Thursday', 'Friday']
```

- **On a single line :**
  - Semicolon (;) allows multiple statements on single line
  - Neither statement should start a new code block



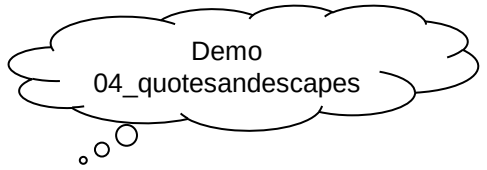
Quotes in Python

- **Python uses quotes to denote string literals :**
  - single ('), double (") and triple (''' or """)
  - Same type of quote should start and end the string
  - Triple quotes can be used to span the string across multiple lines

```
print ('hello' + " " + '''world''')
word = 'word'
sentence = "This is a sentence."
paragraph = """This is a paragraph. It is made up of
                multiple lines and sentences."""
```

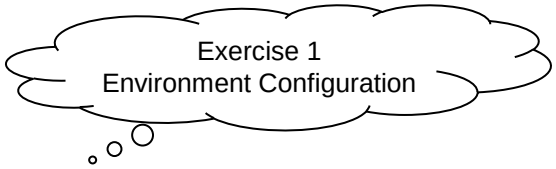
- **Single quote in a single quoted string :**
  - Displayed with an escape character \

```
'What  \'s your name'
```



Summary : Python Intro

- **Python is a general purpose high-level scripting language :**
  - It is interpreted, interactive and object oriented
- **Python was developed by Guido van Rossum around 1990 at :**
  - National Research Institute for Mathematics and Computer Science in Netherlands
- **Python can be started in three different ways :**
  - Interactive Interpreter, Script from Command Line, from inside an IDE, Jupyter Notebook
- **Python is a case sensitive programming language :**
  - `myVar` and `Myvar` are different
- **Python uses line indentation to denote blocks of code :**
  - No braces for blocks of code for class and function definitions or flow control
- **Python allows the use of the line continuation character (\) :**
  - Denote that the line should continue
- **Python uses quotes to denote string literals :**
  - single ('), double (") and triple (''' or """)



© copyright : spiraltrain@gmail.com

SpiralTrain
developer courses

Course Schedule

- Python Intro
- **Variables and Data Types**
- Data Structures
- Control Flow and Operators
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes
- Optional : Python Libraries

- **Variables**
- **Everything is an Object**
- **Multiple Assignment**
- **Strong Typing**
- **Python Numbers**
- **Type Conversions**
- **Conversion Functions**
- **Built-in Number Functions**
- **Python Strings**
- **String Operators**
- **String Operations**
- **Escape Characters**
- **Raw and Unicode Strings**
- **String Formatting**
- **Format Method**
- **Triple Quotes**
- **Built-in String Functions**

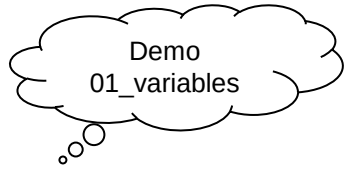
© copyright : spiraltrain@gmail.com

Variables

- **Variables represent memory locations to store values :**
  - When a variable is created memory space is reserved
- **Variables can have different data types like integers, decimal or string :**
  - Based the data type of a variable, the interpreter allocates memory
  - Type of the variable can change and is found with `type` function
- **Variable `x` represents an object :**
  - Unique identifier of an object is an integer and is returned by `id(x)`
- **Assigning values to variables :**
  - Python variables do not have to be explicitly declared to reserve memory space
  - Declaration happens automatically when you assign a value to a variable
- **Equal sign `=` is used to assign values to variables :**
  - Operand to the left of the = operator is the name of the variable
  - Operand to the right of the = operator is the value stored in the variable

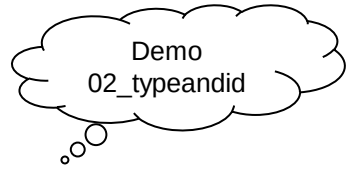
```
counter = 100          # An integer assignment
miles   = 1000.0       # A floating point
name    = "John"       # A string
```

- **Python language does not have constants**



Everything is an Object

- **Each object `x` has :**
  - Unique identifier, an integer, returned by `id(x);`
  - Type returned by `type(x)`
  - Value may or may not be changeable :
    - Value can itself consist of one or more objects
  - Zero or more methods provided by type object to get and/or change content
  - Zero or more names
- **Objects have individuality :**
  - Multiple names in multiple scopes can be bound to the same object
  - This is known as aliasing in other languages
- **Variable names are not properties of an object :**
  - They are stored in separate namespaces
  - Dictionaries of name → object reference (id) pairs



Multiple Assignment

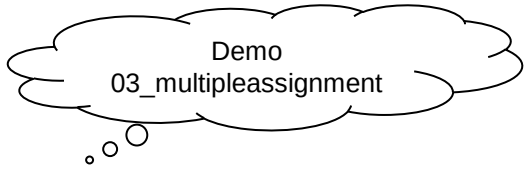
- **Simultaneous assignment of values :**

```
a = b = c = 1
```

- **Integer object is created with value 1 :**
  - All three variables are assigned to the same memory location
- **As soon as one variable changes new object is created :**
  - Changed variable points to this new object
- **In fact once Integer object with value is created :**
  - Value is a constant object that cannot be changed
  - Change leads to the creation of a new object
- **Assignment of multiple objects to multiple variables :**

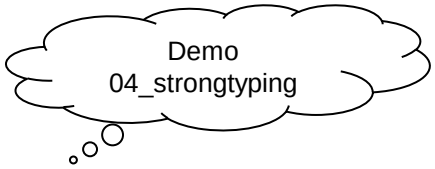
```
a, b, c = 1, 2, "john"
```

- **Two integer objects with values 1 and 2 :**
  - Assigned to variables a and b
- **One string object with the value "john" :**
  - Assigned to the variable c



Strong Typing

- **Data stored in memory can be of many types :**
  - Persons age is stored as a numeric value
  - Persons address is stored as alphanumeric characters
- **A type (or class) of a variable determines :**
  - Operations possible on the variable
  - Storage method for the variable
- **Python is a strongly typed language :**
  - Trying to perform operations on a type that are not allowed gives error
- **Important Python data types :**
  - Numbers
  - String
  - List
  - Tuple
  - Set
  - Dictionary

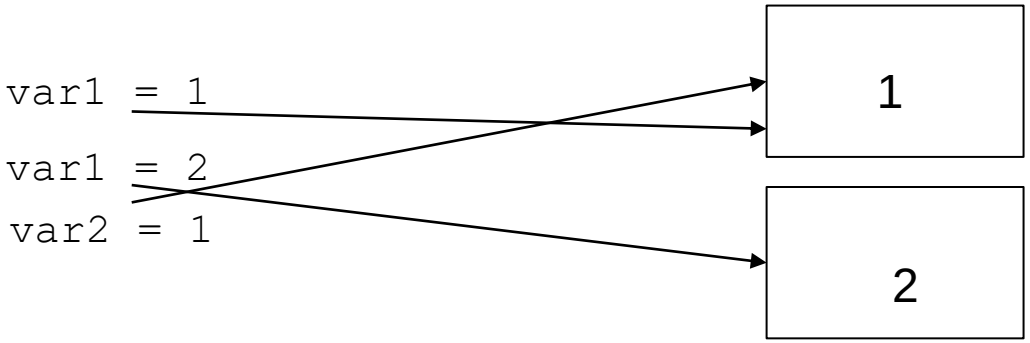


Python Numbers

- **Number objects are created when you assign a value to them :**

```
var1 = 1
var2 = 10
```

- **Numbers are immutable data types :**
  - Changing the value of a number data type results in a newly allocated object

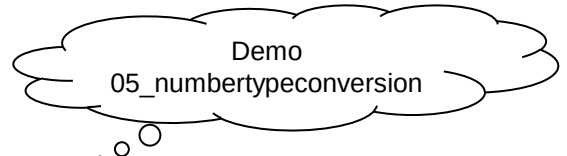


- **Delete single object or multiple objects with `del` statement :**

```
del var
del var_a, var_b
```

Type Conversions

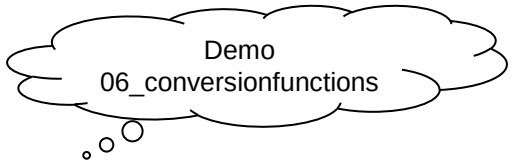
- **Sometimes number must be explicitly converted :**
  - Coerced from one type to another to satisfy requirements of function parameter
- **Built-in functions perform conversion between data types :**
  - `int(x [,base]) :`
    — Converts x to a plain integer, base specifies the base if x is a string
  - `float(x) :`
    — Converts x to a floating-point number
  - `bool([x]) :`
    — Convert value to Boolean using standard truth testing procedure
- **All integers, positive and negative behave as `True` :**
  - Only 0 behaves as `False`
- **All non empty strings behave as `True` :**
  - Only empty string `""` behave as `False` and `None` as well
- **Boolean type is a subtype of the integer type :**
  - Boolean values behave like values 0 and 1
  - When converted to a string in string context `"False"` or `"True"` are returned



Conversion Functions

Name	Description
`str(x)`	Converts object x to a string representation
`repr(x)`	Converts object x to an expression string
`eval(expression)`	Evaluates a Python expression and returns result
`chr(x)`	Converts an integer Unicode codepoint to a character
`ord(x)`	Converts character to integer representing Unicode code point
`hex(x)`	Converts an integer to a hexadecimal string
`oct(x)`	Converts an integer to an octal string

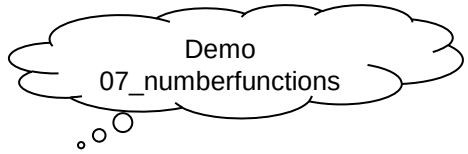
```
y2 = repr('abc')
print (y2)        =>  'abc'
print (type(y2))
y3 = str('abc')
print (y3)        =>  abc
print (type(y3))
i = 65
print (chr(i))     => A
print (hex(i))     => 0x41
print (ord('A'))   => 65
```



Built-in Number Functions

Name	Description
`abs(x)`	Absolute value of x: the (positive) distance between x and zero
`ceil(x)`	The ceiling of x: the smallest integer greater than or equal to *x*
`exp(x)`	The exponential of x: e^x
`fabs(x)`	The absolute value of x, is part of math module
`floor(x)`	The floor of x: the largest integer not greater than x
`log(x)`	The natural logarithm of x, for x> 0
`log10(x)`	The base-10 logarithm of x for x> 0
`pow(x, y)`	The value of x**y
`sqrt(x)`	The square root of x for x > 0
`max(x, y, ..)`	The maximum of a list of numbers
`min(x, y, ..)`	The minimum of a list of numbers

- **For some of these functions `math` module must be imported**



Python Strings

- **Strings are sets of characters between quotation marks :**
  - Pairs of single ' quotes, of double " quotes and of triple (''' or """) quotes supported
  - There is no distinction between single and double quoted strings
- **Creating strings is as simple as assigning a value to a variable :**

```
var1 = 'Hello World!'
var2 = "Python Programming"
```

- **Python does not support a character type :**
  - Characters are treated as strings of length one, thus also considered a substring
  - To access substrings, use square brackets for slicing along with the index

```
print ("var2[1:5]: ", var2[1:5])
```

  - Indexes start at 0 in beginning of the string, then upwards and to -1 at the end
- **Strings are immutable data types like Numbers :**
  - Changing the value of a string data type results in a newly allocated object

String Operators

Operator	Description
`+`	Concatenation. Adds values on either side of the operator
`*`	Repetition. Creates string by concatenating multiple copies of same string
`[]`	Slice. Gives the character from the given index
`[:]`	Range Slice. Gives the characters from the given range
`in`	Membership. Returns true if a character exists in the given string
`not in`	Membership. Returns true if a character does not exist in the given string
`r/R`	Raw String. Suppresses actual meaning of escape characters. Operator for raw strings r or R placed immediately before start quotes of the string
`%`	Format. Performs string formatting

String Operations

- **Examples :**

```
print(str1)           # Prints complete string
print(str1[0])        # Prints first character of the string
print(str1[-1])       # Prints last character of the string
print(str1[2:5])      # Prints characters starting from 3rd to 6th
print(str1 * 2)       # Prints string two times
print(str1 + "TEST")  # Prints concatenated string
print(str1[::-1])     # Prints string backwards
```

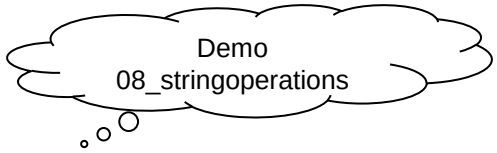
- **String updated by assigning part of it to new string :**
  - New value can be related to its previous value or to a completely different string

```
var1 = 'Hello World!'
var2 = var1[:6] + 'Python'
print("Updated String :- ", var2)
```

- **Result :**

```
Updated String :-  Hello Python
```

- **New string `var2` will have different `id`**



Escape Characters

- **Escape or non-printable characters :**
  - Characters that can be represented with backslash notation

Backslash Character	Hexadecimal Character	Description
\a	0x07	Bell or alert
\b	0x08	Backspace
\cx		Control-x
\e	0x1b	Escape
\f	0x0c	Formfeed
\n	0x0a	Newline
\nnn		Octal notation, n in range from 0-7
\r	0x0d	Carriage return
\s	0x20	Space
\t	0x09	Tab
\v	0x0b	Vertical Tab

Raw and Unicode Strings

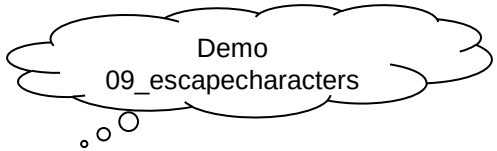
- **Raw String don't treat backslash as a special character :**
  - Every character in a raw string stays the way it was
- **In a normal String the backslash can be escaped :**

```
print ('C:\\nowhere')
```

- **Result :** `C:\nowhere`
- **A raw string is made with the `r` in front of the String :**

```
print (r'C:\\nowhere')
```

- **Result :** `C:\\nowhere`
- **Since Python 3 all strings are stored as Unicode internally :**
  - In Unicode strings consist of (32-bit) code points
  - Allows more characters, including characters from most languages in the world
- `unicodedata` **module :**
  - Provides access to Unicode Character Database (UCD)
  - UCD contains character properties for all Unicode characters



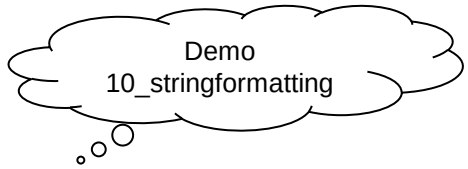
String Formatting

- **String formatting has seen a lot of changes between Python versions :**
  - Earlier version used string formatting using format specifiers like in C
  - Also available came the `format` method to implement string formatting
  - Latest Python versions support `f` strings with variable interpolation like in PHP
- **Operator `%` is string format operator an is unique to strings :**
  - Used in Pythons version of C's `printf()` function

```
print ("My name is %s and weight is %d kg!" % ('Albert', 55))
```

  - Result : `My name is Albert and weight is 55 kg!`
- **In Python 3 new style of formatting available with `format()` function :**
  - Allows complex variable substitutions and value formatting

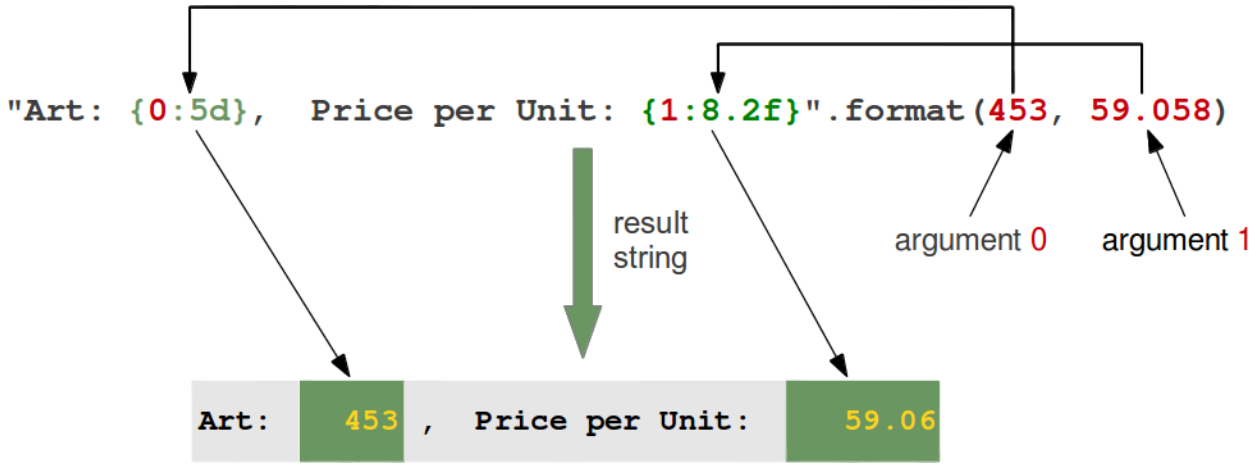
```
s = '{0}, {1}, {2}'.format('a', 'b', 'c')
print (s) => 'a, b, c'
s = '{}, {}, {}'.format('a', 'b', 'c')
print (s) => 'a, b, c'
s = '{2}, {1}, {0}'.format('a', 'b', 'c')
print (s) => 'c, b, a'
```

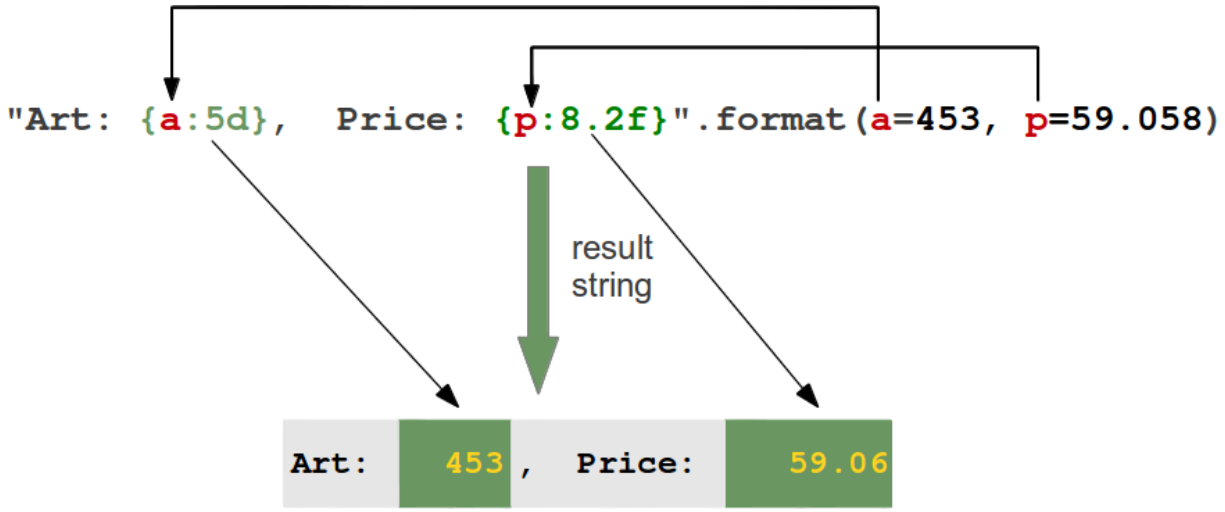


String Format Symbols

Symbol	Functionality
`%s`	String conversion via `str()` prior to formatting
`%i`	Signed decimal integer
`%d`	Signed decimal integer
`%o`	Octal integer
`%x`	Hexadecimal integer
`%f`	Floating point real number
`*`	Argument specifies width or precision
`-`	Left justification
`+`	Display the sign
`<sp>`	Leave a blank space before a positive number
`#`	Add the octal leading zero (‘0’) or hexadecimal leading ‘0x’ of ‘0X’
`0`	Pad from left with zero’s instead of spaces
`%`	‘%%’ leaves you with a single literal ‘%’
`(var)`	Mapping variable (dictionary arguments)
`m.n.`	m is minimum total width and n is nr. of digits to display after decimal point

Format Method





Demo
11_formatmethod

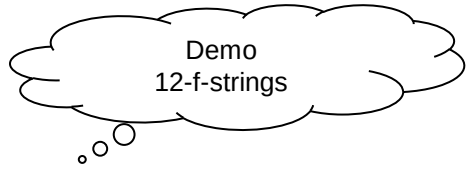
f Strings

- **As of Python 3.6 f-strings are great new way to format strings :**
  - They more readable, faster, more concise and less prone to error than other formatting ways
- **f-strings are string literals that have an `f` at the beginning :**
  - Inside there are curly braces containing expressions that will be replaced with their values
  - Expressions are evaluated at runtime and then formatted using the `__format__` protocol
- **Syntax is similar to the one used with `str.format()` but less verbose :**

```
name = "Albert"
age = 55
print(f"My name is {name} and my age is {age}")
```

- **f-strings are evaluated at runtime, can put valid Python expressions in them :**
  - And can also call methods

```
name = "Enrico Fermi"
f"{name.upper()} is the creator of the first fission reactor"
f"{name.upper():40} is the creator of the first fission reactor"
```



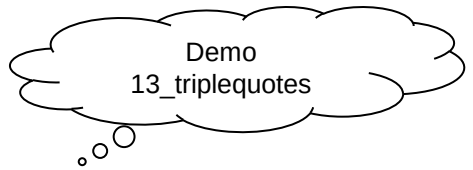
Triple Quotes

- **Allow strings to span multiple lines :**
  - Including verbatim NEWLINEs, TABs, and any other special characters
- **Syntax consists of three consecutive single or double quotes :**

```
para_str = """this string is made up of several lines and non-printable
   characters such as TAB ( \t ) and they will show up that way when displayed.
   NEWLINEs within the string, whether explicitly given like this within the
   brackets [ \n ], or just a NEWLINE
    within the variable assignment will also show up."""
print (para_str)
```

- **Result :**

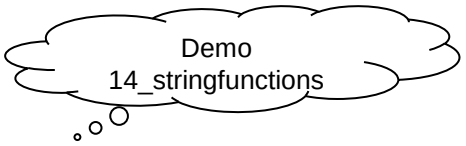
```
this string is made up of several lines and non-printable
characters such as TAB (    ) and they will show up that way when
displayed. NEWLINEs within the string, whether explicitly given like
this within the brackets [
 ], or just a NEWLINE
within the variable assignment will also show up.
```



Built-in String Functions

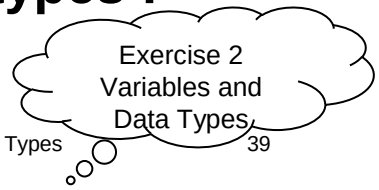
Name	Description
`capitalize()`	Capitalizes first letter of string
`isdigit()`	Returns true if string contains only digits and false otherwise
`isnumeric()`	Returns true if string has only numeric characters, false otherwise
`len(string)`	Returns the length of the string
`lower()`	Converts all uppercase letters in string to lowercase
`upper()`	Converts lowercase letters in string to uppercase
`lstrip()`	Removes all leading whitespace in string
`maketrans()`	Returns a translation table to be used in translate function
`max(str)`	Returns the max alphabetical character from the string str
`min(str)`	Returns the min alphabetical character from the string str
`rstrip()`	Removes all trailing whitespace of string
`swapcase()`	Inverts case for all letters in string
`strip([chars])`	Performs both lstrip() and rstrip() on string

- **Most functions are members of the `String` class :**
  - Must be called using dot notation : `"albert".upper())`



Summary : Variables and Data Types

- **Variables represent memory locations to store values :**
  - When a variable is created memory space is reserved
- **Assigning Values to Variables :**
  - Python variables do not have to be explicitly declared to reserve memory space
- **Equal sign = is used to assign values to variables :**
  - Operand to the left of the = operator is the name of the variable
- **Python has five standard data types :**
  - Numbers, String, List, Tuple, Dictionary
- **Python supports different types to store numeric values :**
  - `int, float, complex`
- **Python boolean type is a subtype of integer :**
  - Only allowed values are 0 and 1 printed as `"True"` and `"False"`
- **Python has built-in functions to convert between data types :**
  - Functions return new object representing converted value



Course Schedule

- Python Intro
- Variables and Data Types
- **Data Structures**
- Control Flow and Operators
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes
- Optional : Python Libraries

- **Sequences**
- **Lists**
- **Accessing Lists**
- **Delete List Elements**
- **Multidimensional Lists**
- **List Operations**
- **List Functions and Methods**
- **Tuples**
- **Usage of Tuples**
- **Tuple Functions**
- **Bytes and Byte Arrays**
- **Sets and Dictionaries**
- **Dictionary Characteristics**
- **Accessing Values in Dictionaries**
- **Properties of Dictionary Keys**
- **Non Mutable Keys**
- **Dictionary Methods**

© copyright : spiraltrain@gmail.com

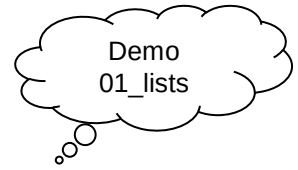
Sequences

- **Most basic data structure in Python is the sequence :**
  - Each element of a sequence is assigned a number - its position or index
  - First index is zero, the second index is one, and so forth
  - Python distinguishes sequences on the basis of mutability
- **Immutable sequences :**
  - Contain objects and content cannot change once created :
    - Objects may references to other objects and these other objects may be changed
  - Examples are Strings (Unicode items), Tuples, Bytes (bytes items)
- **Mutable sequences :**
  - Contain objects and content can be changed once created
  - Examples are Lists and Byte Arrays
- **Operations that work on all sequences types :**
  - Indexing, slicing, multiplying, and checking for membership
- **Python has built-in functions for :**
  - Finding the length of a sequence and its largest and smallest elements

Lists

- **Lists are mutable sequences and similar to arrays in C :**
  - List contains items separated by commas and enclosed within square brackets `[]`
  - Difference with arrays in other languages is that list items can be of different data type
- **Values are accessed using the index or slice operator `[ ]` and `[ : ]` :**
  - Indexes start at 0, then upwards and with end-1
  - Lists can be sliced, concatenated and so on
- **Lists respond to the `+` and `*` operators much like strings :**
  - Meaning is concatenation and repetition
  - Except that result is a new list, not a string
- **Lists respond to all sequence operations usable on strings :**

```
list1 = [ 'abcd', 786 , 2.23, 'john', True ]
print(list1)          # Prints complete list
print(list1[0])       # Prints first element of the list
print(list1[1:3])     # Prints elements starting from 2nd to 4th
print(list1 * 2)      # Prints list two times
print(list1[::-1])    # Prints complete list in reverse order
```



Accessing Lists

- **Use index operator `[]` to access list at specific position :**
  - On left-hand side of assignment operator for given element a value
  - On right-hand side of assignment operator for reading value of element

```
li = ['physics', 'chemistry', 'biology', 'computer science']
print("Value available at index 2 : ")
print(li[2])
li[2] = 'mathematics'
print("New value available at index 2 : ")
print(li[2])
```

- **Elements can be added to a list with `append()` :**

```
list2 = [1, 2, 3]
list2.append(4)
list2.append(5)
print(list2)      # [1, 2, 3, 4, 5]
```

- **Elements can be added to a list at a position with `insert(index, item)` :**

```
list2.insert(0,99)
print(list2)      # [99, 1, 2, 3, 4, 5]
```

Delete List Elements

- **Remove a list element with `del` or `remove()`**
- **Use `del` statement if you know index of element to remove :**

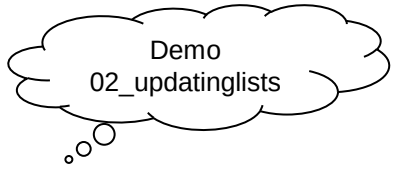
```
li = ['physics', 'chemistry', 'biology', 'computer science']
del li[2]
print("After deleting value at index 2 : ")
print(li) # ['physics', 'chemistry', 'computer science']
```

- **Use `remove()` if want to remove element by value :**

```
li.remove('physics')
print("After deleting value 'physics' ")
print(li)  # ['chemistry', 'computer science']
```

- **List itself can also be removed with `del` :**

```
del list1
list1.append(100)   # gives NameError, name list1 not defined anymore
```



Multidimensional Lists

- **Multidimensional lists created by using other lists as elements :**

```
multilist = [['a', 'b', 'c'], [1, 2, 3, 4]]

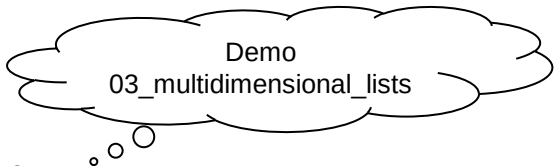
# first dimension
print(multilist[0][0])  # a
print(multilist[0][1])  # b
print(multilist[0][2])  # c

# second dimension
print(multilist[1][0])  # 1
print(multilist[1][1])  # 2
print(multilist[1][2])  # 3
print(multilist[1][3])  # 4

# print (multilist[1][4]) # IndexError: list index out of range
```

- **Delete from a multidimensional list :**

```
multilist = [['a', 'b', 'c'], [1, 2, 3, 4]]
del multilist[0][2]                 # [['a', 'b'], [1, 2, 3, 4]]
multilist.remove(['a', 'b', 'c']) # [[1, 2, 3, 4]]
```

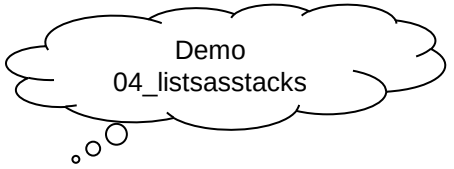


List Operations

Python Expression	Results	Description
`len([1,2,3])`	`3`	Length
`[1,2,3]+[4,5,6]`	`[1,2,3,4,5,6]`	Concatenation
`['go']*3`	`['go','go','go']`	Repetition
`3 in [1,2,3]`	`True`	Membership
`for x in [1,2,3]:` `  print x`	`1 2 3`	Iteration

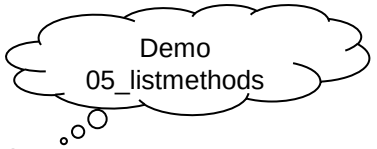
- **Assuming the following input list :** `list1 = ['one','two','three']`

Python Expression	Results	Description
`list1[2]`	`'three'`	Offsets start at zero
`list1[-2]`	`'two'`	Negative : count from the right
`list1[1:]`	`['two', 'three']`	Slicing fetches sections



List Functions and Methods

Name	Description
`len(list)`	Gives the total length of the list
`max(list)`	Returns item from the list with max value
`min(list)`	Returns item from the list with min value
`list(seq)`	Converts a tuple into list
`list.append(obj)`	Appends object obj to list
`list.count(obj)`	Returns count of how many times obj occurs in list
`list.extend(seq)`	Appends the contents of seq to list
`list.index(obj)`	Returns the lowest index in list that obj appears
`list.insert(index, obj)`	Inserts object obj into list at offset index
`list.pop(obj=list[-1])`	Removes and returns last object or obj from list
`list.remove(obj)`	Removes object obj from list
`list.reverse()`	Reverses objects of list in place
`list.sort([key[,reverse])`	sort the items of list in place



Tuples

- **Tuple is an immutable sequence of Python objects :**
  - Consist of a number of values separated by commas
  - Tuples are sequences, just like lists, but tuples are faster because read-only
- **Main differences between lists and tuples :**
  - Lists are enclosed in brackets `[ ]` and their contents and size can be changed
  - Tuples are enclosed in parentheses `( )` and cannot be updated
- **Create tuple by assigning comma-separated values to variable :**
  - Best practice is to put these comma-separated values between parentheses

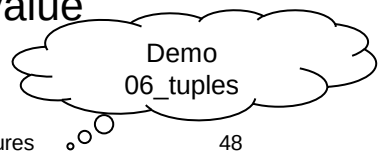
```
tup1 = ('physics', 'chemistry', 'biology', 'computer science')
tup2 = (1, 2, 3, 4, 5 )
tup3 = "a", "b", "c", "d"  # not recommended
```

- **Empty tuple is written as two parentheses containing nothing :**

```
tup1 = ()
```

- **For tuple containing a single value include a comma :**
  - Otherwise it will not be recognized as tuple but is considered a single value

```
tup1 = (50,)
```



Accessing Tuples

- **All of the general sequence operations can be applied on tuples :**
  - Use square brackets for slicing along with index to obtain value, indices start at 0
  - Use the + and * operators for concatenation and repetition

```
tup = (1, 2, 3, 4, 5, 6, 7 )
print(tup[0])       # 1
print(tup[1:5])     # [2, 3, 4, 5]
print(len(tup))      # 7
```

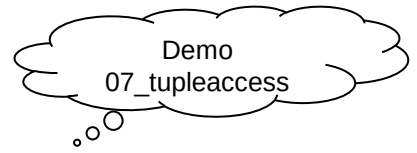
- **Tuples are immutable and can be thought of as read-only lists :**
  - Cannot update them or change values of tuple elements

```
tuple[2] = 1000   # Invalid syntax with tuple
```

- **Portions of existing tuples can be used to create new tuples :**

```
tup1 = (12, 34.56)
tup2 = ('abc', 'xyz')
tup3 = tup1 + tup2
print(tup3)       # (12, 34.56, 'abc', 'xyz')
```

- **Tuples can have other tuples or lists as elements**



Delete Tuple Elements

- **Removing individual tuple elements is not possible :**
  - Another tuple with undesired elements discarded must be created
- **Entire tuple can be removed with `del` statement :**

```
tup1 = ('physics', 'chemistry', 'biology', 'computer science')
print(tup)   # ('physics', 'chemistry', 'biology', 'computer science’)
del tup
print("After deleting tup : ")
print(tup)
```

- **Result after deleting tup :**

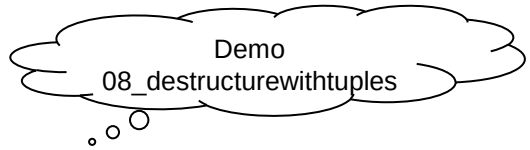
```
Traceback (most recent call last):
  File "test.py", line 9, in <module>
    print tup;
NameError: name 'tup' is not defined
```

- **Error since after `del tup` tuple is gone**

Usage of Tuples

- **Tuples have many uses :**
  - Coordinate pairs, records from a database
- **Any set of comma-separated objects default to tuples :**
  - Event without identifying symbols like parentheses
- **Tuples are faster than lists :**
  - It is faster to loop through a tuple than through a list
- **Tuples can make your code safer :**
  - Allows you to “write-protect” data that does not need to be changed
- **A tuple can have lists as items :**
  - The items in the lists are mutable however
- **Tuple can be destructured and contents assigned to variables :**

```
t = ('Albert', 'Einstein')
print(t)      # ('Albert', 'Einstein')
v, s = t
print(v)      # 'Albert'
print(s)      # 'Einstein'
```



Tuple Functions

- **Only functions and methods that do not modify tuple available**

Name	Description
`len(tuple)`	Gives the total length of the tuple
`max(tuple)`	Returns item from the tuple with max value
`min(tuple)`	Returns item from the tuple with min value
`tuple(seq)`	Converts a list into tuple
`tup.index(i)`	index of the first occurence of i in tup
`tup.count(i)`	total number of occurences of i in tup

Bytes and Byte Arrays

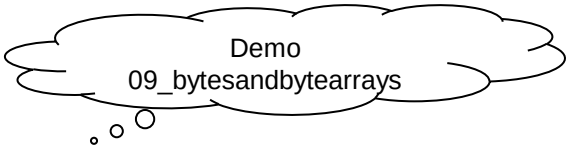
- **Bytes object is an immutable array :**
  - Items are 8-bit bytes, represented by integers in the range `0 <= x < 256`
- **Bytes objects are constructed with :**
  - Bytes literals like `b'abc'` or the built-in function `bytes()`
- **Characters in string become Unicode values in bytes array :**

```
str = "ABCDE"
arr = bytes(str, 'utf-8') # string with encoding 'utf-8'
print(arr)
print(arr[0])  # 65
```

- **Bytes objects can be decoded to strings via `decode()` :**

```
str2 = arr.decode("utf-8")
print (str2)   # ABCD
```

- **Bytearray object is mutable array created `bytearray()` constructor :**
  - Has same interface as immutable bytes objects
- **Extension modules `array` and `collections` :**
  - Provide additional mutable sequence examples



Sets

- **Data type representing unordered collection with no duplicates :**
  - Distinguish between mutable sets and immutable sets
- **Mutable set created by `set()` constructor or by using curly braces `{ }`:**

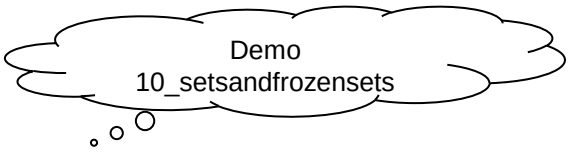
```
s1 = {1, 1, 2, 2}
print(s1)   # {1, 2}
```

- **`set()` constructor expects one iterable object :**

```
s2 = set([3, 3, 4, 4])
print(s2)   # {3, 4}
```

  - Can be modified afterwards by several methods like `add()`
  - Immutable set created by `frozenset()` constructor
- **Supports mathematical operations like union and intersection :**

```
a = set('abracadabra')
b = set('alacazam')
print(a)          # unique letters in a
                  # {'a', 'r', 'b', 'c', 'd'}
print(a - b)      # letters in a but not in b
                  # {'r', 'd', 'b'}
```



Dictionaries

- **Dictionaries consist of pairs of keys and values :**
  - Keys can be almost any immutable Python type but usually numbers or strings
  - Values can be any arbitrary Python object
- **Python 's dictionaries are of hash table type :**
  - Like associative arrays in Perl and PHP
  - Consist of key-value pairs separated by comma's
- **Dictionaries are enclosed by curly braces `{ }` :**

```
dict1 = {'Albert': 2341, 'Robert': 9102, 'Enrico': 3258}
print(dict1) #{'Albert': 2341, 'Robert': 9102, 'Enrico': 3258}
```

- **Values can be assigned and accessed using square brackets `[]` :**

```
dict1['Werner'] = 3490
dict1[2] = "Data"
print(dict1) # {'Albert': 2341, 'Robert': 9102, 'Enrico': 3258, 'Werner': 3490, 2: 'Data'}
```

- **Dictionaries have no concept of order among elements :**
  - Elements are never "out of order", they are simply unordered

Accessing Values in Dictionary

- **Each key is separated from its value by : a colon :**
  - Items are separated by comma's and whole dictionary is enclosed in curly braces
  - Empty dictionary without any items is written with just two curly braces : `{}`
- **Keys are unique within a dictionary while values may be the same :**
  - Values of a dictionary can be of any type
  - Keys must be of an immutable data type such as strings, numbers or tuples
- **Use square brackets along with the key to obtain its value :**

```
print(dict1["Robert"])  # 9102
print(dict1[2])         # Data
```

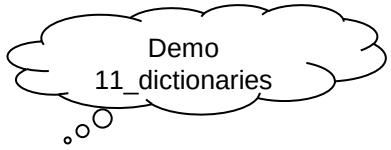
- **Dictionary can be updated by modifying an existing entry :**

```
dict1['Enrico'] = 9999
print(dict1['Enrico'])  # 9999
```

- **Dictionary entry can be deleted :**

```
del dict1['Albert']
print(dict1['Albert'])    # KeyError
```

- **Entire contents of dictionary is cleared with `clear()` :**
  - Entire dictionary can be deleted in single operation with `del`



Properties of Dictionary Keys

- **Dictionary values have no restrictions :**
  - Can be any arbitrary Python object
  - Either standard objects or user-defined objects
- **There two important points to about dictionary keys :**
  - No duplicate keys allowed
  - Keys must be immutable
- **In case duplicate keys are encountered during assignment :**
  - Last assignment wins

```
dict1 = {'Albert': 2341, 'Robert': 9102, 'Albert': 3258}
print (dict['Albert'])    # Result : 3258
```

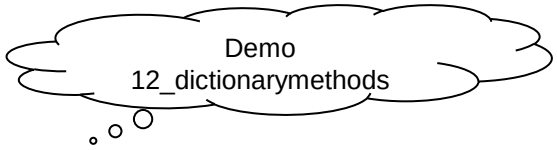
- **Strings, numbers, or tuples can be dictionary keys :**
  - But something like `['Name']` is not allowed since a list is mutable

```
dict = {['Albert']: 2341, 'Robert': 9102}
print (dict[['Albert']])
```

- **Result :** `TypeError: list objects are unhashable`

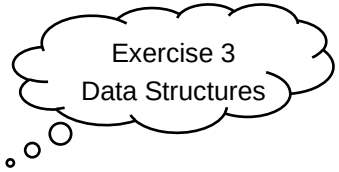
Dictionary Functions and Methods

Name	Description
`len(dict)`	Gives the total length or nr. of items in the dictionary.
`str(dict)`	Produces a printable string representation of a dictionary
`fromkeys()`	Create new dictionary with keys from seq , values set to value
`dict.clear()`	Removes all elements of dictionary dict
`dict.copy()`	Returns a shallow copy of dictionary dict
`dict.get(key)`	For key key, returns value or default if key not in dictionary
`dict.popitem()`	Remove and return arbitrary (key, value) pair from dictionary
`dict.items()`	Returns a list of dict's (key, value) tuple pairs
`dict.keys()`	Returns list of dictionary dict's keys
`dict.update(dict)`	Adds dictionary dict's key-values pairs to dict
`dict.values()`	Returns list of dictionary dict's values



Summary : Data Structures

- **Sequences are most basic data structure :**
  - Each element of a sequence is assigned a number - its position, or index
  - First index is zero, the second index is one, and so forth
- **Lists are mutable sequences :**
  - Contain items separated by commas and enclosed within square brackets `[]`
  - Similar to arrays in C but list items can be of different data type
- **Tuples are immutable sequences :**
  - Enclosed in parenthesis ( ) instead of brackets
  - Consist of a number of values separated by commas
- **Sets are unordered collections with no duplicates :**
  - Support mathematical operations like union and intersection
  - Set is mutable and `frozenset` is immutable
- **Dictionaries are comparable to associative arrays :**
  - Pairs of keys and values separated by comma , and colon :
  - Enclosed by curly braces `{ }`, values access with square brackets `[]`



© copyright : spiraltrain@gmail.com

Course Schedule

- Python Intro
- Variables and Data Types
- Data Structures
- **Control Flow and Operators**
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes
- Optional : Python Libraries

- **Control Flow Constructs**
- **if Statement**
- **else Statement**
- **elif Statement**
- **Nested if**
- **while Loop**
- **Infinite while Loop**
- **for Loop**
- **Iterating by Sequence Index**
- **break Statement**
- **continue Statement**
- **Loop with else Combination**
- **pass Statement**
- **Python Operators**
- **Python Operator Precedence**

© copyright : spiraltrain@gmail.com

Control Flow Constructs

- **Every programming has control flow constructs :**
  - Sequential constructs like statements
  - Branching constructs to take a certain direction
  - Iteration construct to repeat code a number of times
- **Branching Statements :**
  - `if`
  - `if...else`
  - `elif`
- **Iteration Statements :**
  - `for`
  - `while`
  - `continue`
  - `pass`

if Statement

- `if` **statement in Python is similar to that of other languages :**
  - Contains a logical expression using which data is compared
  - Decision is made based on the result of the comparison
- **Syntax** `if` **statement :**

```
if expression:
    statement(s)
```

- **Expression is evaluated first :**
  - If true that is then statement(s) block are executed
  - Otherwise next statement following statement(s) block is executed
- **Numbers and strings represent true values :**
  - Except number 0 and empty string
- **Grouping statements :**
  - Statements indented by the same number of character spaces :
    - Considered to be part of a single block of code
  - Python uses indentation as its method of grouping statements

if Example

```
var1 = 100
if var1:
   print ("1 - Got a true expression value")
   print (var1)

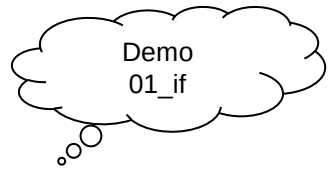
var2 = 0
if var2:
   print ("2 - Got a true expression value")
   print (var2)
print ("Good bye!")
```

- **Result :**

```
1 - Got a true expression value
100
Good bye!
```

- **In case `if` has only one line it may go on same line :**

```
if ( expression == 1 ) : print ("Value of expression is 1")
```



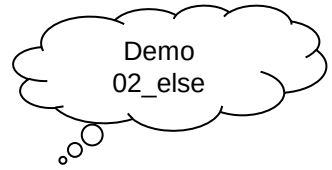
else Statement

- **Is combined with an `if` statement :**
  - Contains code that executes in case the `if` statement resolves to 0 or false
- `else` **statement is an optional statement :**
  - Only one `else` statement can follow the `if` statement
- **Syntax `if...else` statement :**

```
if expression:
   statement(s)
else:
   statement(s)
```

- **Example asking input with `input` and `endswith` methods :**

```
name = input('What is your name? ')
if name.endswith('Gumby'):
    print ('Hello, Mr. Gumby')
else:
    print ('Hello, stranger')
```

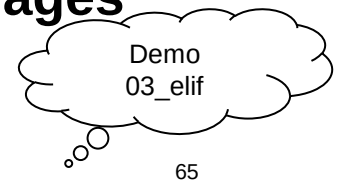


elif Statement

- **Allows checking of multiple expressions for truth value :**
  - Execute a block of code as soon as one of the conditions evaluates to true
- `elif` **statement is optional :**
  - Arbitrary number of `elif` statements may be following an `if`
- **Syntax of the** `if...elif` **statement :**

```
if expression1:
   statement(s)
elif expression2:
   statement(s)
elif expression3:
   statement(s)
else:
   statement(s)
```

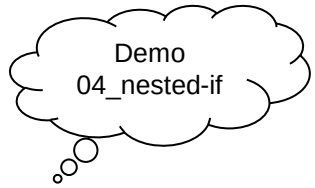
- **No support for** `switch` **or** `case` **statements as in other languages**



Nested if...elif...else

- `if...elif...else` **construct in other** `if...elif...else` **construct :**
  - Allows checking another condition after condition resolves to true
- **Syntax nested** `if...elif...else` **:**

```
if expression1:
   statement(s)
   if expression2:
      statement(s)
   elif expression3:
      statement(s)
   else
      statement(s)
elif expression4:
   statement(s)
else:
   statement(s)
```



while Loop

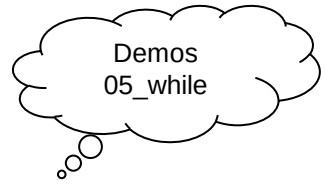
- **Control flow construct to repeat code number of times :**
  - Repetition continues until conditional expression becomes false
  - Expression must be a logical expression evaluating to true or false

```
while expression:
    statement(s)
```

- `while` **loop ends when conditional expression becomes false :**
  - Control is passed to the statement following the loop
- **In case the condition of a** `while` **never resolves to a false value :**
  - Result is an infinite loop
- **Infinite loop might be useful in client/server programming :**
  - Server runs continuously on an open port so that clients can communicate with it
- **If** `while` **clause has only one statement :**
  - Placement on the same line is allowed

```
while expression : statement
```

- **Python does not support** `do .. while` **construct**



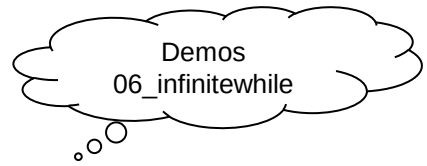
Infinite while Loop

```
var = 1
while var == 1 :  # This constructs an infinite loop
   num = input("Enter a number  :")
   print("You entered: ", num)
print("Good bye!")
```

- **Result :**

```
Enter a number  :20
You entered:  20
Enter a number  :3
You entered:  3
Enter a number between :Traceback (most recent call last):
  File "test.py", line 5, in <module>
    num = input("Enter a number :")
KeyboardInterrupt
```

- **Above example will go in an infinite loop :**
  - Use CTRL+C to stop the program



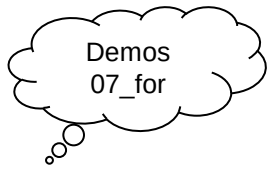
for Loop

- **Control flow construct to repeat code number of times :**
  - Iterates over the items of any sequence, such as a list or a string
- **Syntax :**

```
for var in sequence:
   statements(s)
```

- `for` **loop operation :**
  - First item in the sequence is assigned to iterating variable `var`
  - Next statements block is executed
  - Each item in the list is assigned to `var`
  - Statements(s) block is executed until the entire sequence is finished

```
words = ['one', 'way', 'ticket', 'to', 'mars']
for word in words:
    print(word , end=' ')
print('')
list1 = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
for number in list1:
    print(number)
```



Iterating by Sequence Index

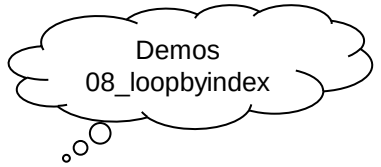
- **Also possible is to iterate by index into the sequence :**

```
planets = ['venus', 'mars',  'jupiter']
for index in range(len(planets)):
   print('Current planet :', planets[index])
print("Lift off!")
```

- **Result :**

```
Current planet : venus
Current planet : mars
Current planet : jupiter
Lift off!
```

- **Used utility functions are built-in `len()` and `range()` functions :**
  - `len()` provides the total number of elements in the tuple
  - `range()` gives us the actual sequence to iterate over

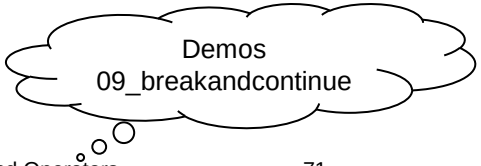


break and continue Statements

- `break` **statement terminates the current loop iteration :**
  - Resumes execution at next statement after the loop
- `continue` **statement returns control to the beginning of the loop :**
  - Remaining statements in the current iteration of the loop are skipped
  - Control is moved back to the top of the loop
- `break` **and** `continue` **can be used in both** `while` **and** `for` **loops**

```
for letter in 'Python':
   if letter == 'h':
      continue
   print('Current Letter :', letter) # letter 'h' is not printed
print("After first loop!")

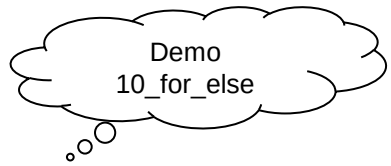
for letter in 'Programming':
   if letter == 'i':
      break
   print('Current Letter :', letter) # loop stops on letter 'i'
print("After second loop!")
```



Loop with else Combination

- `else` **statement can be associated with loop statements**
- **If** `else` **statement is used with a** `for` **loop :**
  - `else` statement executed when loop has exhausted iterating the list but not on a break
- **If** `else` **statement is used with a** `while` **loop :**
  - `else` statement is executed when the condition becomes false
- **In following example :**
  - `else` statement is combined with `for` statement
  - Search is done for a perfect square in the range 99 tot 80, number 81 is found

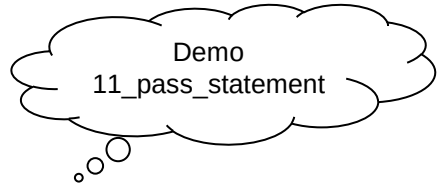
```
from math import sqrt
for n in range(99, 80, -1):
    root = sqrt(n)
    print(root)
    print(int(root))
    if root == int(root):
       print("Perfect square is %d" % (n)))
       break
else:
    print("Didn't find it!")
```



pass Statement

- **Used when a statement is required syntactically :**
  - But it is not desired that any command or code executes
- `pass` **statement is a null operation :**
  - Nothing happens when it executes
  - Also useful in places where code has not been written yet e.g. in stubs
- `pass` **statement is helpful when :**
  - A code block is created but it is no longer required
  - Statements inside the block may then be removed
  - Block may remain with `pass` statement so it doesn't interfere with other code parts

```
for letter in 'Python':
   if letter == 'h':
      pass
      print('This is pass block')
   print('Current Letter :', letter)
print("Good bye!")
```



Python Operators

- **Operators perform operations on the operands :**

  ```
  4 + 5 = 9
  ```

- **Here 4 and 5 are called operands and `+` is called operator**

  ```
  3 ** 2 =  9
  ```

- **Here 3 and 2 are called operands and `**` is called operator**
- **Python language supports a number of traditional operators :**
  - Arithmetic Operators
  - Comparison Operators
  - Logical (or Relational) Operators
  - Assignment Operators
  - Conditional (or ternary) Operators
- **Python also has operators for data structures and objects :**
  - Membership operators test for membership in strings, lists and tuples
  - Identity operators compare memory locations of two objects
- **Python does not support `++` and `--` operators**

Python Arithmetic and Assignment Operators

Operator	Arithmetic Operator Description
+	Adds values on either side of the operator
-	Subtracts right hand operand from left hand operand
*	Multiplies values on either side of the operator
/	Divide left hand operand by right hand operand with decimal result
//	Division with quotient result with digits after decimal point removed
%	Divides left hand operand by right hand operand, returns remainder
**	Performs exponential (power) calculation on operators

Operator	Assignment Operator Description
=	Assigns values from right side operands to left side operand
+=	Adds right operand to left operand and assigns result to left operand
-=	Subtracts right operand from left operand and assigns result to left operand
*=	Multiplies right operand with left operand and assigns result to left operand
/=	Divides left operand by right operand and assigns result to left operand
%=	Takes modulus using two operands and assigns result to left operand

Python Comparison and Bitwise Operators

Operator	Comparison Operator Description
==	Checks if values of two operands are equal or not. If yes condition is true
!=	Checks if values of two operands are equal or not. If not condition is true
<>	Same as != operator
>	Checks if value of left operand is greater than right operand
<	Checks if value of left operand is less than right operand
>=	Checks if value of left operand is greater than or equal to right operand
<=	Checks if value of left operand is less than or equal to right operand

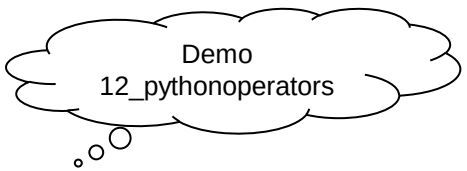
Operator	Bitwise Operator Description
&	Copies a bit to the result if it exists in both operands
|	Copies a bit to the result if it exists in one or both of the operands
^	Copies a bit to the result if it is set in one but not both operands
~	Unary Binary Ones Complement Operator. Has effect of flipping the bits
<<	Left operands value is moved left by nr. of bits specified by right operand
>>	Left operands value is moved right by nr. of bits specified by right operand

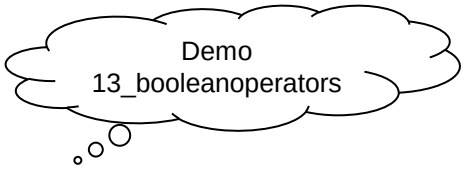
Logical, Membership and Identity Operators

Operator	Logical Operator Description
`and`	If both operands are true then the condition becomes true
`or`	If any of the operands are non-zero then the condition becomes true
`not`	Reverses logical state of operand. True condition will become false

Operator	Membership Operator Description
`in`	Evaluates to true if it finds variable in specified sequence, false otherwise
`not in`	Evaluates to true if it does not find a variable in sequence, false otherwise

Operator	Identity Operator Description
`is`	Evaluates to true if variables on both sides point to the same object
`is not`	Evaluates to true if variables on both sides do not point to the same object



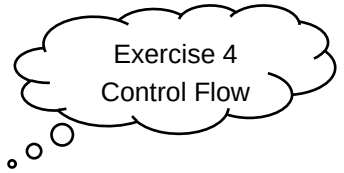


Python Operators Precedence

Operator	Description
`**`	Exponentiation, raise to the power
`~ + -`	Complement, unary plus and minus
`* / % //`	Multiply, divide, modulo and floor division
`+ -`	Addition and subtraction
`>> <<`	Right and left bitwise shift
`&`	Bitwise ‘AND’
`^|`	Bitwise exclusive ‘OR’ and regular ‘OR’
`<= < > >=`	Comparison operators
`<> == !=`	Equality operators
`= %= /= //= -= += |= etc`	Assignment operators
`is, is not`	Identity operators
`in, not in`	Membership operators
`not, or, and`	Logical operators

Summary : Control Flow and Operators

- **Python has the following control flow constructs :**
  - `if, if...else, elif, loop, while, continue, pass`
- **Pythons `if` statement test a logical expression for `True` or `False` :**
  - `else` may be combined with an `if` statement
  - `elif` allows checking of multiple expressions
- **Pythons `while` loop is used to repeat code number of times :**
  - Repetition continues until conditional expression becomes false
- **Pythons `for` loop is used to repeat code number of times :**
  - Iterates over the items of any sequence, such as a list or a string
- **Pythons `pass` statement used when syntax requires statement :**
  - Nothing happens when it executes
- **Pythons `break` and `continue` statements :**
  - Used to terminate current iteration and pass control elsewhere
- **Python supports the following operators :**
  - Arithmetic, Comparison, Logical, Assignment and Conditional Operators



© copyright : spiraltrain@gmail.com

Course Schedule

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow and Operators
- **Functions**
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes
- Optional : Python Libraries

- **Functions**
- **Function Syntax**
- **Calling Functions**
- **Pass by Value**
- **Pass by Reference**
- **Overwriting References**
- **Function Arguments**
- **Keyword Arguments**
- **Default Arguments**
- **Variable Length Arguments**
- **Anonymous Functions**
- **Syntax Lambda Functions**
- **return Statement**
- **Scope of Variables**

© copyright : spiraltrain@gmail.com

Functions

- **Blocks of reusable code performing some action :**
  - Python has many built-in functions like `print()`,`input()`,`len()` etc.
  - It is also possible to create user-defined functions
- **Functions have the following benefits :**
  - Modularity of applications and code reuse
- **Rules to define a function in Python :**
  - Function blocks begin `def` followed by function name and parentheses
  - Any input parameters or arguments should be placed within parentheses
  - It is also possible to define parameters inside these parentheses
- **Code block within every function :**
  - Starts with a colon (:) and is indented
- **Statement `return [expression]` exits a function :**
  - Optionally passing back an expression to the caller
  - A return statement with no arguments is the same as return `None`

Function Syntax

- **First statement of function can be documentation string of function :**
  - docstrings are picked up by documentation tool `pdoc`
  - Installed by `pip3 install pdoc3` and used with `pdoc –html <function of module>`

```
def functionname( parameters ):
   """function_docstring"""
   function_statements
   return [expression]
```

- **By default :**
  - Parameters have a positional behavior
  - Should pass them in the same order that they were defined
- **Example of a simple Python function :**
  - Takes a string as input parameter and prints it on standard screen

```
def showmessage( str ):
   """This prints a passed string into this function"""
   print(str)
   return
```

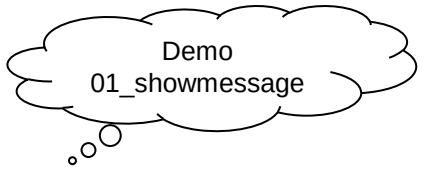
Calling Functions

- **Execute functions by calling them :**
  - From another function or directly from the Python prompt
  - If functions specify parameters, it is required to pass them
- **Function must be defined before they can be called :**
  - Order of definition and call is important
- **Example of calling the previously defined `showmessage()` function :**

```
showmessage("First call to user defined function!")
showmessage("Again second call to the same function")
showmessage()  # results in error
```

- **Result :**

```
First call to user defined function!
Again second call to the same function
Traceback (most recent call last):
  File "01_showmessage.py", line 8, in <module>
    showmessage()
TypeError: showmessage() takes exactly 1 argument (0 given)
```



Pass by Value

- **Python passes function parameters by value :**
  - Parameter in function is copy of variable in caller and refers to same object
  - Change to object via parameter in function is seen by variable in function's caller
  - Mutable and immutable objects behave differently

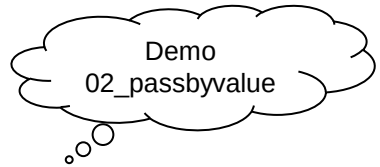
```
def changeme( mylist ):
   mylist.append(40)
   print("List inside the function: ", mylist)
   return
```

- **Call to `changeme` function :**

```
mylistouter = [10,20,30]
changeme(mylistouter)
print("List outside the function: ", mylistouter)
```

- **Reference of passed object is used to append values :**

```
List inside the function changeme1:  [10, 20, 30, 40]
List outside the function:  [10, 20, 30, 40]
```



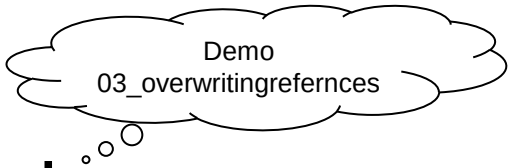
Overwriting References

- **Reference may be overwritten in the function :**

```
def changeme(mylist):
   mylist = [1,2,3,4] # This would assign new reference
   print("Values inside the function: ", mylist)
   return
```

- **Call `changeme` function :**

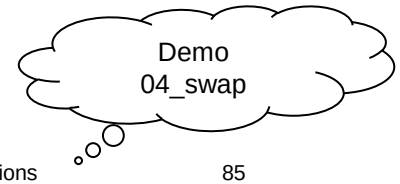
```
mylistouter = [10,20,30]
changeme(mylistouter)
print("Values outside the function: ", mylistouter)
```



- **Parameter `mylist` is changed in the function `changeme` :**
  - Changing `mylist` within the function does not affect `mylistouter`
  - Function does nothing to original list and finally this would produce following result :

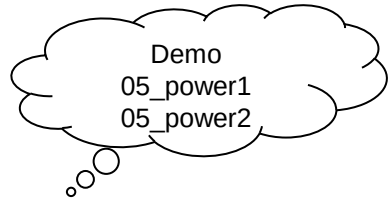
```
Values inside the function:  [1, 2, 3, 4]
Values outside the function:  [10, 20, 30]
```

- **Changing immutable references :**
  - Always results in new objects and is true for Strings and Numbers



Function Arguments

- **Functions can be called with different types of arguments**
- **Required arguments :**
  - Passed to a function in correct positional order
  - Number of arguments in call should match exactly with the definition
- **Keyword arguments :**
  - Caller identifies arguments by name and allows parameters to be out of order
  - Python interpreter is able to use the keywords to match the values with parameters
- **Default arguments :**
  - Assume default value if value is not provided in the function call for that argument
- **Variable-length arguments :**
  - Sometimes more arguments must be processed than specified in function definition
  - These are called variable-length arguments and are not named in function definition
- **Python does not support function overloading :**
  - Defining same method with different parameters
  - Python is dynamically typed and supports optional arguments



Keyword Arguments

- **Function definition :**

```
def printinfo(name, age):
   """This prints name and age passed into this function"""
   print("Name: ", name)
   print("Age: ", age)
   return
```

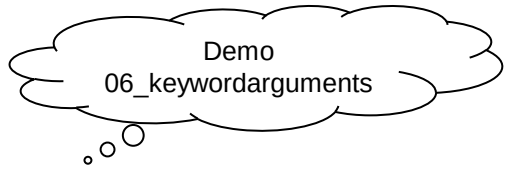
- **Function call with keyword arguments :**

```
printinfo(age=55, name= "Albert")
```

- **Result :**

```
Name:  Albert
Age:  55
```

- **Calling code specifies arguments by name :**
  - Parameters do not have to follow the order in the function definition



Default Arguments

- **Function definition :**

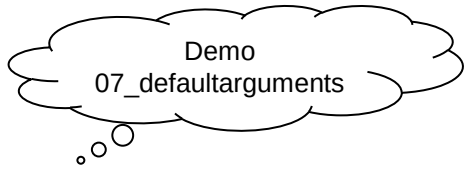
```
def printinfo(name, age = 35):
    """This prints name and age passed into this function"""
    print("Name: ", name)
    print("Age: ", age)
    return
```

- **Functions calls with and without default arguments :**

```
printinfo(age=55, name= "Albert")
printinfo(name= "Werner")
```

- **Results :**

```
Name:  Albert
Age:  55
Name:  Werner
Age:  35
```



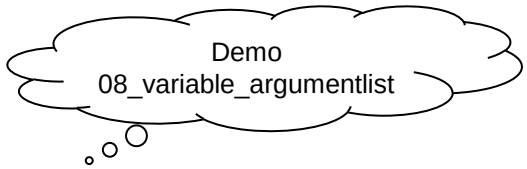
Variable Length Arguments

- **Syntax for function with variable arguments :**

```
def testArgLists(arg, *argtuple, **kwargs):
    """Function referencing argument, argument list and keyword arguments"""
    print('arg:', arg)
    print('args:', argtuple)
    print('kwargs:', kwargs)
```

- `*args_tuple` :
  - Asterisk (*) indicates variable holding values of all non-keyword variable arguments
  - Tuple remains empty if no additional arguments are specified during function call
- `**args_keywords` :
  - Double asterisk (**) indicates a variable-length argument list with keywords
  - Dictionary remains empty if no keyword arguments are passed

```
testArgLists(1, 2, 3, 4, 5, x=1, y=2)
testArgLists(33, x=8, y=9)
testArgLists(44)
```



Anonymous Functions

- `lambda` **keyword is used to create small anonymous functions :**
  - Not declared in the standard manner by using the `def` keyword
- **Lambda form can take any number of arguments :**
  - Return just one value in the form of an expression
  - Cannot contain commands or multiple expressions
- **Characteristics of lambda functions :**
  - Have their own local namespace
  - Cannot access variables besides in parameter list and in global namespace
  - Cannot be a direct call to print because lambda requires an expression
  - Can be stored in variables and passed and returned to functions
- **Lambda's appear to be a one-line versions of a function :**
  - However they are not equivalent to inline statements in C or C++

Syntax Lambda Functions

- **Lambda functions contain only a single statement :**

```
lambda [arg1 [,arg2,.....argn]]:expression
```

- **Function definition :**

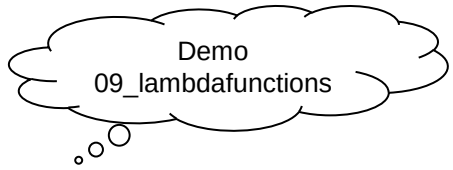
```
summation = lambda arg1, arg2: arg1 + arg2
```

- **Function call :**

```
print("Value of total : ", summation( 10, 20 ))
print("Value of total : ", summation( 20, 20 ))
```

- **Result :**

```
Value of total :  30
Value of total :  40
```



return Statement

- `return [expression]` **exits a function :**
  - Optionally passing back an expression to the caller
  - `return` statement with no arguments is the same as return `None`
  - `return` statement is not required and then default return value is `None`
- **Function definition :**

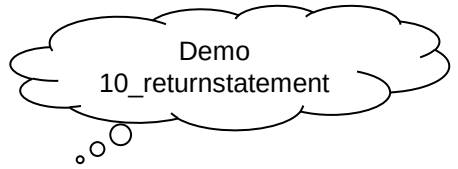
```
def summation(arg1, arg2):
   """Add both the parameters and return them"""
   tot = arg1 + arg2
   print("Inside the function : ", tot)
   return tot
```

- **Function call :**

```
total = summation(10, 20)
print("Outside the function : ", total)
```

- **Result :**

```
Inside the function :  30
Outside the function :  30
```



Scope of Variables

- **Accessibility of variable depends on where it is declared :**
  - Scope determines where in program you can access a particular identifier
- **There are two basic scopes of variables in Python :**
  - Global variables
  - Local variables
- **Global versus Local variables :**
  - Variables defined inside a function body have a local scope
  - Variables defined outside have a global scope
- **Local variables :**
  - Can be accessed only inside the function in which they are declared
  - When you call a function, the variables declared inside it are brought into scope
- **Global variables :**
  - Can be accessed throughout the program body by all functions
  - Refer to global instead of a local variable inside function with `global` keyword

Example Variable Scope

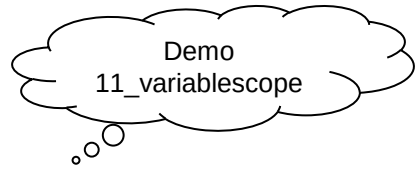
```
x = 50 # This is a global variable
y = 50 # This is a global variable
```

- **Function definitions :**

```
def func1():
    x = 2
    print ('Changed local x to', x)    # prints 2
def func2():
    print('Access global x as', x)
def func3():
    global y
    y = 2
    print ('Changed global y to', y)    # prints 2
```

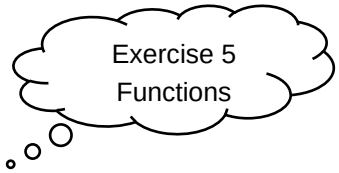
- **Function calls :**

```
func1()                                 # prints 2
print('x is still', x)                  # prints 50
func2()                                 # prints 50
print('x is still', x)                  # prints 50
func3()                                 # prints 2
print('Value of y is', y)               # prints 2
```



Summary : Functions

- **Functions are blocks of reusable code performing some action :**
  - Python has built-in functions and it is possible to create user-defined functions
- **Python functions are defined with the `def` statement :**
  - They can accept parameters, have a code block and a return statement
- **Python passes function parameters by value :**
  - Parameter in function refers to same object as in caller
- **Functions can be called with different types of arguments :**
  - Required, Keyword, Default and Variable-length arguments
- **`lambda` keyword is used to create small anonymous functions :**
  - Return value as an expression, cannot contain commands or multiple expressions
- **Local variables :**
  - Variables defined inside a function body have a local scope
- **Global variables :**
  - Variables defined outside functions have a global scope



© copyright : spiraltrain@gmail.com

Course Schedule

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- **Modules**
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- Python Classes
- Optional : Python Libraries

- **Modules**
- **import Statement**
- **from…import Statement**
- **Locating Modules**
- **Creating and Using Modules**
- **dir Function**
- **Python Packages**
- **Explict Import Modules**
- **Implicit Import Modules**
- **globals and locals Functions**
- **reload Function**
- **Test Harnass**
- **Installing Packages**
- **Virtual Environments**

© copyright : spiraltrain@gmail.com

Modules

- **Used to organize number of Python definitions in single file :**
  - Used to logically organize your Python code
  - Can be bind and referenced
- **Module is a file consisting of Python code :**
  - Defines functions, classes, and variables
  - May also include runnable code
  - Grouping related code into a module makes code easier to understand and use
- **Python code for module `test` resides in a file named `test.py` :**
  - Name of the module is the name of the file without extension
- `test.py`:

```
def print_func( par ):
   print ("Hello : ", par)
   return
```

- **Modules allows code reuse :**
  - Code in module can be reused in other python file

import Statement

- `import` **statement followed by module name :**
  - Makes functions and classes from module available for use in file issuing `import` statement
- `import` **has the following syntax :**

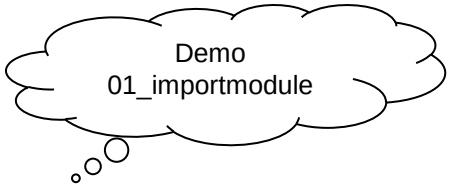
```
import module1[, module2[,... moduleN]
```

- **On encountering an** `import` **statement :**
  - Module is imported if the module is present in the search path
  - Search path is list of directories that interpreter searches for modules to import
  - Module is loaded only once, regardless of the number of times it is imported
- **Import module** `test.py` **with** `import test` **at top of the script :**
  - Use name of the module as prefix when accessing functions, classes or variables

```
import test  # Now you can call defined function
test.print_func('pipo') # Hello : pipo
```

- **Name to reference module can be changed with :**

```
import test as t
t.print_func('pipo')    # Hello : pipo
```



from...import Statement

- `from...import` **makes specific names from module accessible:**

```
from modname import name1[, name2[, ... nameN]]
```

- **Function** `sqrt` **from module** `math` **is imported with :**

```
from math import sqrt
```

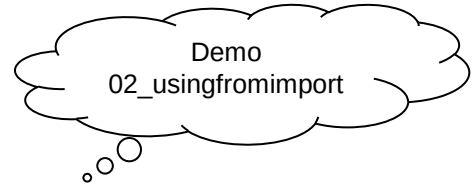
- **Statement does not import entire module** `math` **:**
  - Only item `sqrt` from module `math` ends up in global symbol table of importing module
- **Now the function** `sqrt` **can be used without prefix :**

```
print(sqrt(9))  # 3.0
```

- **Also possible is to import all names from a module with :**

```
from modname import *
```

- **Statement should be used sparingly :**
  - More names are then visible which can give rise to name conflicts



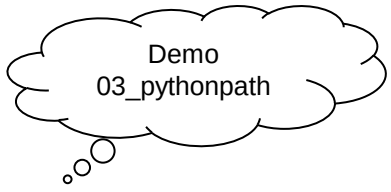
Locating Modules

- **On import Python interpreter searches for module as follows :**
  - Current directory
  - Each directory in the shell variable `PYTHONPATH`
  - Default path which is OS dependent
- **Search path is stored in system module `sys` as `sys.path` variable :**
  - Should import `sys` to be able to access path
  - `sys` module give also access to command line arguments or parameters to script
- `PYTHONPATH` **variable :**
  - Environment variable, consisting of a list of directories
  - Syntax of `PYTHONPATH` is same as that of the shell variable `PATH`
- **Typical `PYTHONPATH` from Windows system :**

```
set PYTHONPATH=c:\python37\lib;
```

- **Typical `PYTHONPATH` from UNIX system :**

```
set PYTHONPATH=/usr/local/lib/python
```



Using Modules

- **Every Python program is already a module :**
  - Modules must have `.py` extension

```
# Filename: testmodules.py
def sayhi():
    print('Hi, this is testmodules speaking.')
version = '0.1'
```

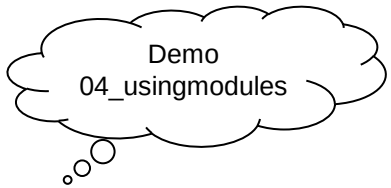
- **Module can be imported by other module if on `sys.path` :**

```
import testmodules
testmodules.sayhi()
print(testmodules.version)
```

- **Alternatively `from .. import` can be used :**

```
from testmodules import sayhi, version
sayhi()
print(version)
```

- **Modules contain :**
  - Variables, functions, classes and potentially executable code



dir Function

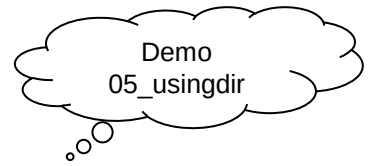
- **Returns sorted list of strings with names accessible by a module :**
  - List contains the names defined in a module and names that are imported
  - Modules, Variables, Functions
- **Example `import` built-in module `math` :**

```
import math
content = dir(math)
print(content)
```

- **Result :**

```
['__doc__', '__name__', '__package__', 'acos', 'acosh', 'asin', 'asinh', 'atan',
'atan2', 'atanh', 'ceil', 'copysign', 'cos', 'cosh', 'degrees', 'e', 'erf',
'erfc', 'exp', 'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum',
'gamma', 'hypot', 'isfinite', 'isinf', 'isnan', 'ldexp', 'lgamma', 'log',
'log10', 'log1p', 'modf', 'pi', 'pow', 'radians', 'sin', 'sinh', 'sqrt', 'tan',
'tanh', 'trunc']
```

- `__name__` **is module's name**
- `__package__` **is package name of module**



Packages in Python

- **Package is way to organize number of modules together as unit :**
  - Package is simply a directory and has a hierarchical file directory structure
  - Consists of modules and subpackages and sub-subpackages and so on
- **Multiple functions in files and different Python classes :**
  - Create packages out of those classes
  - Package provide a namespace and can prevent name classes
- **Typically modules of a package are placed in a directory :**
  - Consider for example `Canon.py`, `Leika.py` and `Nikon.py` in `cameras` directory
- `Canon.py` **could define a simple function like :**

```
def CanonInfo():
    print ("Canon camera")
```

- **Other two files could define similar but different functions :**
  - `def Leika()` and `def Nikon()`
- **Package is required to have** `__init__.py` **in its directory :**
  - Indicates directory contains package and makes functions available when package is imported

Explicit Import of Package Modules

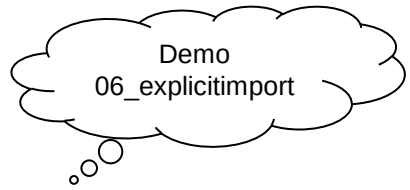
- `__init__.py` **should be present but can be empty**
- **If** `__init.py__` **empty modules of package must be imported explicitly :**

```
import cameras.Canon
import cameras.Leika
import cameras.Nikon
```

- **Functions in package are called with full name :**

```
cameras.Nikon.NikonInfo()
cameras.Canon.CanonInfo()
cameras.Leika.LeikaInfo()
```

- **Full name consists of :**
  - Package name (directory), module name, function name



Implicit Import of Package Modules

- **Following lines can be added to `__init__.py` :**

```
from cameras.Canon import CanonInfo
from cameras.Leika import LeikaInfo
from cameras.Nikon import NikonInfo
```

- **Now modules can be imported by importing the package :**

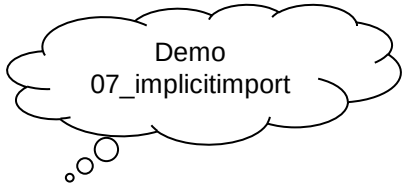
```
import cameras
```

- **Import `cameras` package and call methods as follows :**

```
cameras.NikonInfo()
cameras.CanonInfo()
cameras.LeikaInfo()
```

- **Also `from .. import` can be used :**

```
from cameras import NikonInfo, CanonInfo, LeikaInfo
NikonInfo()
CanonInfo()
LeikaInfo()
```



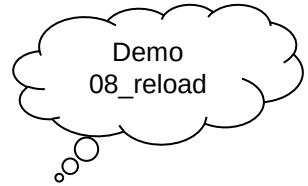
reload Function

- **Reloads a previously imported module :**
  - When module is imported, code in the top-level portion is executed
  - When using multiple imports of the same module top-level code will not execute again
- **To reexecute top-level code in a module :**
  - Use the `reload()` function from `importlib` module
- **Syntax :**

```
reload(module_name)
```

- `module_name` **:**
  - Name of the module you want to reload
  - Not the string containing the module name
- **To reload hello module :**

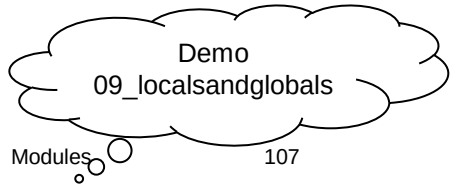
```
reload(hello)
```



globals and locals Functions

- **Namespace is a dictionary containing :**
  - Variable names (keys) and their corresponding objects (values)
- **Variables can be in local namespace or global namespace :**
  - Variables declared outside functions are in the global namespace
  - Variables declared in function are in the local namespace of that function
- `locals()` **and** `globals()` **methods called from within a function :**
  - `locals()` returns all the names that can be accessed locally from that function
  - `globals()` returns all the names that can be accessed globally from that function
- **Return type of both these functions is dictionary :**
  - Names can be extracted using the `keys()` function

```
x = 1
def f() :
    y = 1
    print("Globals from f")
    print(globals().keys())     # dict_keys(['__name__', '__doc__', … 'x', 'f'])
    print("Locals from f")
    print(locals().keys())      # dict_keys(['y'])
f()
```

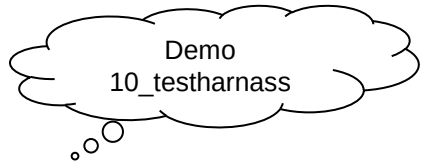


Test Harnass

- **Used to run code only when script is run, not when imported**

```
if __name__ == '__main__' :
```

  - Only true only when the file is run
  - Not when the module is imported
  - Good practice to test how module is used
- **Top level code in module only evaluated first time it is imported :**
  - Python maintains an internal list of all modules that have been imported
- **When you import a module for the first time :**
  - Module script is executed in its own namespace until the end
  - Internal list is updated, and execution of continues after the import statement
- **Can use reload to force new import e.g. when module changed**



Installing Packages

- **From PyPI or from VCS, other indexes, local archives**
- **On Linux or OS X :**
  - `pip install -U pip setuptools`
- **On Windows :**
  - `python -m pip install -U pip setuptools`
- **Common usage of `pip` :**
  - Install from the Python Package Index using a requirement specifier

```
pip install package
pip install package==1.3
pip install package~=1.3
pip install package>1.3
pip install --upgrade package
```

- **Requirements file contains lists of packages to install :**

```
pip install -r requirements.txt
pip list                          # list all packages installed
pip freeze > requirements.txt     # reads packages installed and saves versions in file
pip uninstall package
```

Virtual Environments

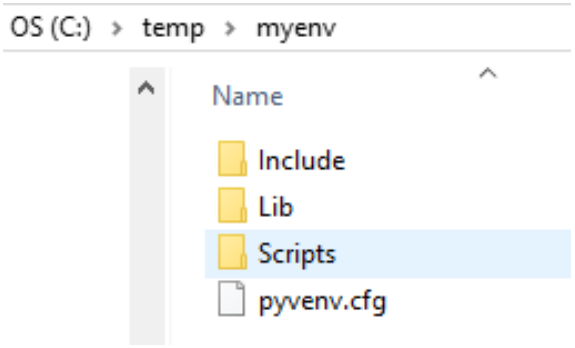
- **Install packages in separate location, not shared globally :**
  - Each environment has its own Python binary
  - Handles different apps requiring different versions of package
- **Applications won't break by subsequent library updates :**
  - Makes it easier to create distributions
- **Create virtual environment as follows :**

```
python -m pip install virtualenv
virtualenv c:\temp\myenv
```

- **Or :** `python -m venv myenv c:\temp\myenv (>Python3.4)`
- **Switching Environment :**
  - Run `activate` from Scripts directory of virtual environment to set path and change prompt
  - Run `deactivate` to return to standard Python environment

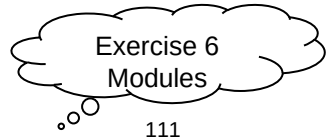
```
C:\temp\myenv\Scripts>activate

(myenv) C:\temp\myenv\Scripts>
```



Summary : Modules

- **Module is a file consisting of Python code :**
  - Defines functions, classes, and variables, but may also contain runnable code
  - Used to logically organize Python code
- **import statement in some other Python source file :**
  - Leads to import if the module is present in the search path
  - Current directory, directories in `PYTHONPATH`, default OS path
- **Namespace is a dictionary containing :**
  - Variable names (keys) and their corresponding objects (values)
  - Variables can be in local namespace or global namespace
- **Functions show names defined in module or namespaces :**
  - `dir`, `locals` and `globals` functions
- **Package defines a single Python application environment :**
  - Has a hierarchical file directory structure
- **Virtual environments allow to install packages in separate location :**
  - Not shared globally and each environment has its own Python binary



© copyright : spiraltrain@gmail.com

SpiralTrain
developer courses

Course Schedule

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- **Comprehensions**
- Exception Handling
- Python IO
- Python Database Access
- Python Classes
- Optional : Python Libraries

- **Functional Programming**
- **Map and Filter**
- **Reduce and Lambda**
- **List Comprehensions**
- **Syntax List Comprehension**
- **Filtered List Comprehension**
- **Syntactic Sugar**
- **Nested List Comprehensions**
- **Comprehension Features**
- **Zip Function**
- **Dictionary Construction with Zip**
- **Dictionary Comprehension**
- **Dictionary from Keys**
- **Set Comprehension**

© copyright : spiraltrain@gmail.com

Functional Programming

- **Programming style that is focused on expressions :**
  - Also called expression oriented programming
- **Python has functions and features enabling functional approach :**
  - `map, filter, reduce, lambda`
  - `list comprehension`
- `map(aFunction, aSequence) :`
  - Applies operation to each item and collects the result
- `filter(aFunction, aSequence) :`
  - Extracts each element in the sequence for which function returns True
- `reduce(aFunction, aSequence) :`
  - Reduces list to single value by combining elements via supplied function
  - In functools in Python 3.0
- `lambda :`
  - Small anonymous function consisting of single line

Map and Filter

- **Higher-order functions `map` and `filter` that operate on lists :**

```
def square(x):
        return x*x
def even(x):
        return 0 == x % 2
some_list = list(map(square, range(10,20)))
print (some_list)
```

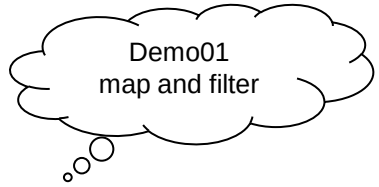
- **Output :** `[100, 121, 144, 169, 196, 225, 256, 289, 324, 361]`

```
some_list = list(filter(even, range(10,20)))
print (some_list)
```

- **Output :** `[10, 12, 14, 16, 18]`

```
some_list = list(map(square, filter(even, range(10,20))))
print (some_list)
```

- **Output :** `[100, 144, 196, 256, 324]`
- **More Pythonic are list comprehensions**



Reduce and Lambda

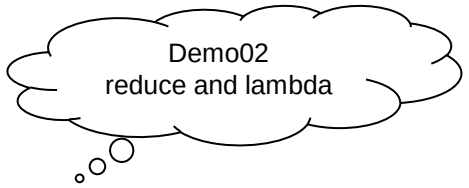
- `reduce` **accepts iterator to process and returns single result :**

```
from functools import reduce
var1 = reduce( (lambda x, y: x * y), [1, 2, 3, 4] )
print (var1) # 24
```

- **Above code works as follows :**
  - `reduce` passes current product along with next item from list to `lambda` function
  - Default first item in sequence initialized starting value
- **Same functionality with `for` loop :**

```
li1 = [1, 2, 3, 4]
result = li1[0]
for x in li1[1:] :
    result = result * x

print (result)
```



List Comprehensions

- **Construct for creating new list based on existing lists :**
  - Languages like Haskell, Erlang, Scala and Python have them
- **Comprehension term :**
  - Comes from math's set comprehension notation to define sets in terms of other sets
- **Powerful and popular feature in Python :**
  - Generate new list by applying operation or function to every member of original list

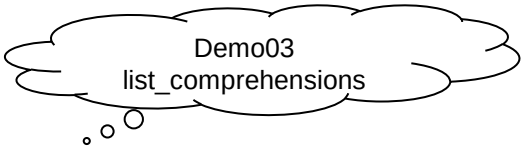
```
slist = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print (slist)

list_str = [str(x) for x in slist]
print (list_str)
```

- **Output :** `['1', '2', '3', '4', '5', '6', '7', '8', '9', '10']`

```
new_list = [x * 2 for x in slist]
print (new_list)
```

- **Output :** `[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]`



Syntax List Comprehensions

- **Syntax of a list comprehension may involve :**
  - `for` loop, an `in` operation and an `if` statement
- **Python's notation :**
  - `[ expression for name in list ]`
  - Expression :
    - Some calculation or operation acting upon the variable name
  - For each member of the list, the list comprehension
    - Sets name equal to that member
    - Calculates a new value using expression
  - Collects these new values into a list which is return value of list comprehension

```
li = [3, 6, 2, 7]
li_new = [elem*2 for elem in li]   => [6, 12, 4, 14]
```

- `if` **statement acts as a filter and is optional :**

```
[x-10 for x in grades if x>0]
```

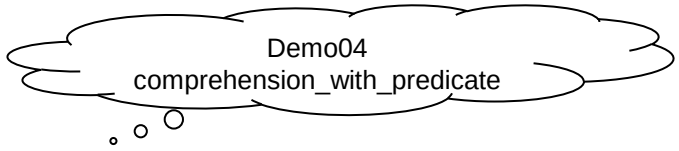
Filtered List Comprehension

- **Filter determines if expression is performed on list member :**
  - For each element of list, checks if it satisfies the filter condition
- **If filter condition returns False :**
  - Element is omitted from the list before the list comprehension is evaluated
- **Traditional syntax :**

```
slist = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
for x in slist:
    if (x % 2) == 0:
        new_list.append(str(x))
print (new_list)  => ['2', '4', '6', '8', '10']
```

- **Pythonic syntax using list comprehension :**

```
new_list = [str(x) for x in slist if (x % 2) == 0]
print (new_list)  => ['2', '4', '6', '8', '10']
```



Syntactic Sugar

- **List comprehensions are syntactic sugar for higher-order functions :**

```
[ expression for name in list  ]
map( lambda name: expression, list )
```

- **Example using higher order function :**

```
lm = list(map( lambda x: 2*x+1, [10, 20, 30]  ))
print (lm)  # [21, 41, 61]
```

- **Example using list comprehension :**

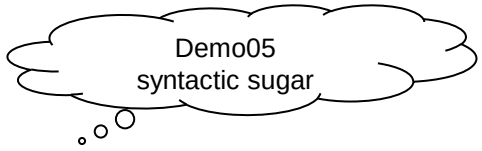
```
lc = [ 2*x+1 for x in [10, 20, 30]  ]
print (lc)   # [21, 41, 61]
```

- **Example using higher order function :**

```
ls1 = list(map( lambda x: 2*x+1, filter( lambda x: x > 0 , [10, 20, 30, -10] )))
print(ls1)
```

- **Example using list comprehension :**

```
ls2 = [ 2*x+1 for x in [10, 20, 30, -10] if x > 0 ]
print (ls2)
```



Nested List Comprehensions

- **List comprehensions can easily be nested :**
  - Take list as input and produce list as output
- **In following inner comprehension produces :** `[4, 3, 5, 2]`

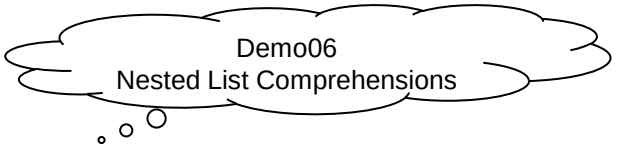
```
li = [3, 2, 4, 1]
[elem*2 for elem in [item+1 for item in li] ]  #[8, 6, 10, 4]
```

- **Another example is :**

```
list_a = ['A', 'B']
list_b = ['C', 'D']
list_c = [[x+y for x in list_a] for y in list_b]
print (list_c)
```

- **Produces :**

```
[['AC', 'BC'], ['AD', 'BD']]
```



Comprehension Features

- **If list contains elements of different types :**
  - Expression must operate correctly on the types of all of list members.
- **If elements of list are other containers :**
  - Name can consist of container of names matching type and “shape” of list members

```
list_d = [('a' , 1), ('b', 2), ('c', 7)]
list_e = [ n * 3 for (x, n) in list_d]
print (list_e)  # [3, 6, 21]
```

- **Containers are objects that contain references to other objects :**
  - Lists, types, dictionaries
- **Expression can also contain user-defined functions :**

```
def subtract(a, b):
    return a - b
oplist = [(6, 3), (1, 7), (5, 5)]
list_f = [subtract(y, x) for (x, y) in oplist]
print (list_f)   # ([-3, 6, 0]
```



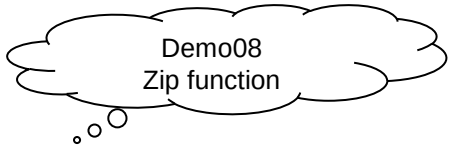
Zip Function

- `zip(*iterables)` :
  - Makes an iterator that aggregates elements from each of the iterables
  - iterator stops when the shortest input iterable is exhausted
- **Used to combine two lists in list of pairs :**

```
ls1 = [1,2,3]
ls2 = [4,5,6]
ls3 = list(zip(ls1,ls2))
print (ls3)   # [(1, 4), (2, 5), (3, 6)]
```

- **Original lists can also be restored using same function :**

```
b1, b2 = zip(*ls3)
print (b1)
print (b2)
```



Dictionary Construction with zip

- `zip` **can be used to generate dictionaries :**
  - When keys and values must be computed at runtime
- **Dictionaries are traditionally created with :**

```
dict1 = {'a':1, 'b':2, 'c':3}
print (dict1)   # {'a': 1, 'c': 3, 'b': 2}
```

- **Or we can make it by assigning values to keys :**

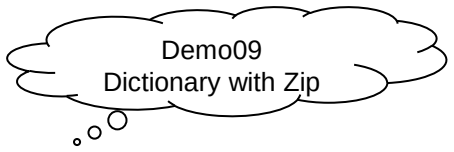
```
dict2 = {}
dict2['a'] = 1; dict2['b'] = 2; dict2['c'] = 3
print (dict2)    # {'a': 1, 'c': 3, 'b': 2}
```

- **Sometimes keys and values are only know at runtime :**
  - Can then use `zip` to create lists and loop through them in parallel like this :

```
dict3 = {}
for (k,v) in zip(keys, values):
    dict3[k] = v
print (dict3)    # {'a': 1, 'b': 2, 'c': 3}
```

- **Alternative is using constructor with :**

```
dict4 = dict(zip(keys, values))
```



Dictionary Comprehensions

- **Dictionary comprehension is like a list comprehension :**
  - Constructs a dictionary instead of a list
- **Dictionary comprehension makes dictionary from two lists :**

```
keys = ['a', 'b', 'c']
values = [1, 2, 3]

dict5 = { k:v for (k,v) in zip(keys, values)}
print (dict5)  # {'a': 1, 'b': 2, 'c': 3}
```

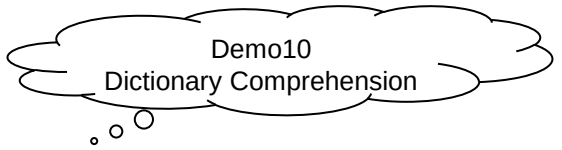
- **Requires actually more code then constructor function :**

```
dict6 = dict(zip(keys, values))
print (dict6)     # {'b': 2, 'c': 3, 'a': 1}
```

- **Many more occasions to use dictionary comprehension :**

```
dict7 = {x: x**2 for x in [1,2,3,4,5]}
print (dict7)   # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

dict8 = {x.upper(): x*3 for x in 'abcd'}
print (dict8)   #{'A': 'aaa', 'C': 'ccc', 'B': 'bbb', 'D': 'ddd'}
```



Dictionary from Keys

- **We can initialize dictionary from `keys` as follows :**

```
dict9 = dict.fromkeys(['a','b','c'], 0)
print (dict9)   # {'a': 0, 'c': 0, 'b': 0}
```

- **Can use dictionary comprehension to do the same thing :**

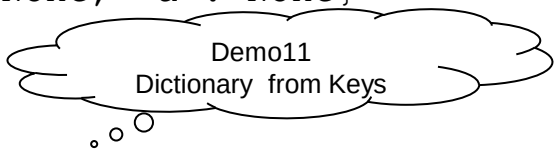
```
dict10 = {k: 0 for k in ['a','b','c']}
print (dict10)  # {'a': 0, 'c': 0, 'b': 0}
```

- **Generate dictionary by iterating each element :**

```
dict11 = dict.fromkeys('dictionary')
print (dict11) # {'d': None, 'o': None, 'i': None,
# 't': None, 'a': None, 'c': None, 'y': None, 'n': None, 'r': None}
```

- **Now using comprehension :**

```
dict12 = {k:None for k in 'dictionary'}
print (dict12) # {'t': None, 'y': None, 'c': None,
# 'd': None, 'i': None, 'r': None, 'o': None, 'n': None, 'a': None}
```



Set Comprehensions

- **Sets have their own comprehension syntax as well :**
  - Quite similar to the syntax for dictionary comprehensions
  - Difference is that sets just have values instead of `key:value` pairs
- **Set comprehensions can take set as input :**
  - Following set comprehension calculates squares of set of numbers from 0 to 9

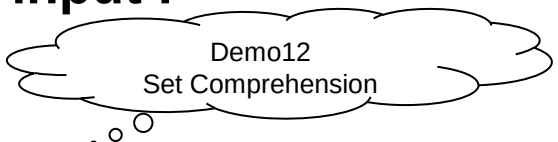
```
my_set = set(range(10))
print (my_set)  # {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
my_set2 = {x ** 2 for x in my_set}
print (my_set2) # {0, 1, 4, 81, 64, 9, 16, 49, 25, 36}
```

- **Set comprehensions can contain `if` clause to filter items :**

```
my_set3 = {x for x in my_set if x % 2 == 0}
print (my_set3)  #{0, 8, 2, 4, 6}
```

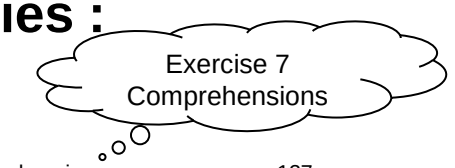
- **Set comprehensions do not have to take set as input :**
  - They can take any sequence

```
my_set4 = {2**x for x in range(10)}
print (my_set4)  #{32, 1, 2, 4, 8, 64, 128, 256, 16, 512}
```



Summary : Comprehensions

- **List comprehension :**
  - Generate new list by applying operation or function to every member of original list
- **Syntax of a list comprehension may involve :**
  - `for` loop, an `in` operation and an `if` statement
- **If filter determines if expression is performed on list member :**
  - For each element of list, checks if it satisfies the filter condition
- **List comprehensions are syntactic sugar :**
  - For higher-order functions like `map` and `filter`
- **List comprehensions can easily be nested :**
  - Take list as input and produce list as output
- **Dictionary comprehension is like a list comprehension :**
  - Constructs a dictionary instead of a list
- **Set comprehensions have similar syntax as dictionaries :**
  - Difference is that sets have values instead of `key:value` pairs



© copyright : spiraltrain@gmail.com

SpiralTrain
developer courses

Course Schedule

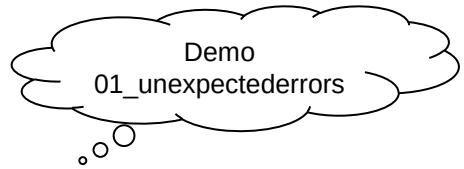
- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- Comprehensions
- **Exception Handling**
- Python IO
- Python Database Access
- Python Classes
- Optional : Python Libraries

- **Unexpected Errors**
- **Exception Handling**
- **Typed Exception Handling**
- **Exception Handling with Else**
- **except Clause Multiple Exceptions**
- **Python Standard Exceptions**
- **try-finally Clause**
- **Example try-finally**
- **Exception Arguments**
- **Raising Exceptions**
- **Example Raising Exceptions**
- **User Defined Exceptions**

© copyright : spiraltrain@gmail.com

Unexpected Errors

- **Python provides exceptions to handle unexpected errors :**
  - Exceptions are events disrupting normal program flow
  - May occur during the execution of program instructions
- **When Python encounters conditions it can't cope with :**
  - An exception is raised
  - An exception is a Python object that represents an error
- **When a Python script raises an exception :**
  - The exception must be handled immediately otherwise the script terminates
- **Another feature to handle unexpected errors are assertions :**
  - Assertions are tests which must be true to continue

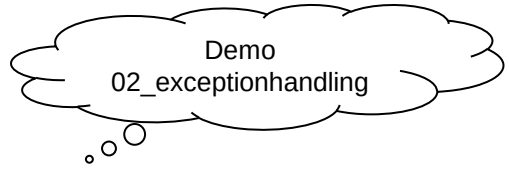


Exception Handling

- **Place code that might raise an exception in a `try` block :**
  - Code in a `try` block is tested for exceptions
- **Include an `except` statement after the `try` block :**
  - Place code to handle the problem as elegantly as possible there
  - Generic `except` clause handles any exception

```
try :
    x = 1
    y = 0
    z = x/y                     # ZeroDivisionError
except:
    print('Cannot divide by zero')
 try :
    fh = open("somefile", "r")  # IOError
except:
    print('Cannot open file')
```

- **After handling exception in `except` statement :**
  - Normal program flow continues

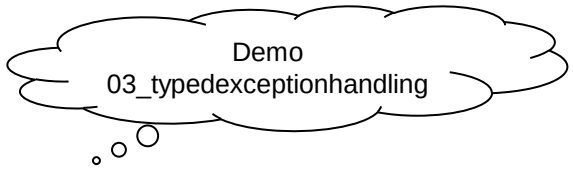


Typed Exception Handling

- **Generic `except` statement with no exceptions defined :**
  - Catches all exceptions that occur and is not considered good programming practice
  - Does not make programmer identify root cause of the problem
- **Better is the usage of `except` clauses with a type :**
  - Single `try` statement can have multiple typed `except` statements
  - Useful when the `try` block might throw different types of exceptions

```
try :
    x = 1
    y = 0
    z = x/y                     # ZeroDivisionError
    tup = (1,2,3)
    tup[0] = 2                  # TypeError
except ZeroDivisionError :
    print ('Division by zero')
except TypeError :
    print ('Cannot change tuple contents')
```

- `except` **corresponding to exception type executes**



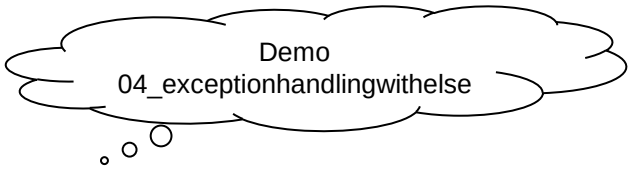
Exception Handling with Else

- **After `except` clause(s) `else` clause can be included :**
  - Executes if the code in the `try:` block does not raise an exception

```
try:
   Do you operations here
except:
   If there is any exception, then execute this block
else:
   If there is no exception then execute this block
```

- **Example :**

```
try:
    x = int(input("Please enter a number: "))
except ValueError:
    print("Oops! That was not valid number")
else:
    print ("Yeah! That was a really good number")
```



except Clause Multiple Exceptions

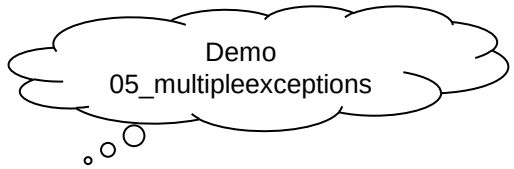
- **Same `except` clause may also handle multiple exceptions :**

```
try:
   Do you operations here;
except(Exception1[, Exception2[,...ExceptionN]]]):
   If there is any exception from the given exception list,
   then execute this block
else:
   If there is no exception then execute this block
```

- **Example :**

```
for value in (None, "Hi!"):
    try:
        print ("Attempting to convert", value, "-->",)
        print (float(value))
    except(TypeError, ValueError):
        print ("Something went wrong!")
```

- **Disadvantage is error messages are less distinctive**

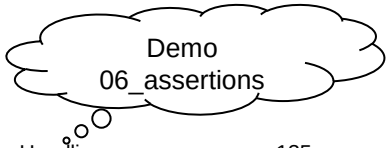


Python Standard Exceptions

Name	Description
`Exception`	Base class for all exceptions
`StopIteration`	Raised when `next()` of iterator does not point to any object
`SystemExit`	Raised by the `sys.exit()` function
`StandardError`	Base class for built-in exceptions except StopIteration and SystemExit
`AritmeticError`	Base class for all errors that occur for numeric calculation
`OverflowError`	When calculation exceeds maximum limit for numeric type
`FloatingPointError`	Raised when a floating point calculation fails
`ZeroDivisionError`	Raised when division by zero occurs for numeric types
`AssertionError`	Raised in case of failure of the Assert statement
`AttributeError`	Raised in case of failure of attribute reference or assignment
`EOFError`	Raised when end of file is reached
`ImportError`	Raised when an import statement fails
`TypeError`	Raised when operation attempted that is invalid for data type
`ValueError`	Raised when function for data type has invalid values
`RuntimeError`	Raised when generated error does not fall into any category

Python Standard Exceptions

Name	Description
`KeyboardInterrupt`	Raised when user interrupts program execution, usually by pressing CTRLC-C
`LookupError`	Base class for all lookup errors
`IndexError`	Raised when an index is not found in a sequence
`KeyError`	Raised when the specified key is not found in the dictionary
`NameError`	Raised when an identifier is not found in the local or global namespace
`UnboundLocalError`	Raised when trying to access a local variable in a function or method but no value has been assigned to it
`EnvironmentError`	Base class for all exceptions that occur outside the Python environment
`IOError`	Raised when an input /output operation fails, such as the print statement or the open() function when trying to open a file that does not exist
`OSError`	Raised for operating system related errors
`SyntaxError`	Raised when there is an error in Python syntax
`IndentationError`	Raised when indentation is not specified properly
`SystemError`	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit



try-finally Clause

- `finally` **block may be used along with a** `try` **block :**
  - `finally` block is always executed
  - Whether or not the try-block raised an exception
- **Syntax of the** `try-finally` **:**

```
try:
    Do you operations here;
    Due to any exception, this may be skipped
finally:
    This would always be executed
```

- **Combining** `finally` **and** `except` **is allowed :**
  - `try` block can be followed by multiple `except` clauses and one `finally` clause
- **Limitation :**
  - It is not possible to use an `else` clause along with a `finally` clause

Example try-finally

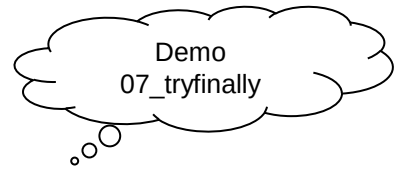
```
try:
   fh = open("testfile", "w")
   fh.write("This is my test file for exception handling!!")
finally:
   print ("Error: can\'t find file or read data")
```

- **Result without permission to open the file in writing mode :**

```
Error: can't find file or read data
```

- **Same example can be written more cleanly as follows :**

```
try:
   fh = open("testfile", "w")
   try:
      fh.write("This is my test file for exception handling!!")
   finally:
      fh.close()
except IOError:
   print ("Error: can\'t find file or read data")
```



Exception Arguments

- **An exception can have an argument :**
  - Value that gives additional information about the problem
  - Contents of the argument vary by exception
- **Capture argument by supplying a variable in `except` clause :**

```
try:
   Do you operations here;
except ExceptionType as Argument:
   You can print value of Argument here...
```

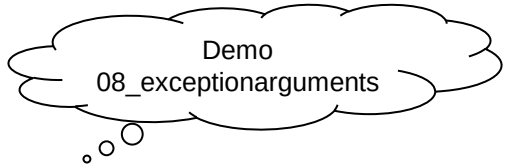
- **In case of handling a single exception :**
  - Variable follows the name of the exception in the except statement
- **In case of handling multiple exceptions :**
  - Variable follows the tuple of the exception
- **Variable will receive the value of the exception :**
  - Can receive a single value or multiple values in the form of a tuple
  - Tuple usually contains the error string, the error number and an error location

Example Argument of an Exception

```
# Define a function here.
def temp_convert(var):
   try:
      return int(var)
   except ValueError as e:
      print ("Argument does not contain numbers\n", e)
# Call above function here
temp_convert("xyz");
```

- **Result :**

```
The argument does not contain numbers
invalid literal for int() with base 10: 'xyz'
```



Raising Exceptions

- **Exceptions can be raised with the `raise` statement :**

```
raise [Exception [, args [, traceback]]]
```

- **Explanation :**
  - Exception is the type of exception, for example `NameError`
  - Argument is a value for the exception argument
  - The argument is optional; if not supplied, the exception argument is None.
  - Final argument `traceback` is optional and is the `traceback` object for exception
- **Example :**
  - An exception can be a string, a class, or an object
  - Most of the exceptions that the Python core raises are classes
    - with an argument that is an instance of the class
- **Defining new exceptions can be done as follows**

Example Raising an Exception

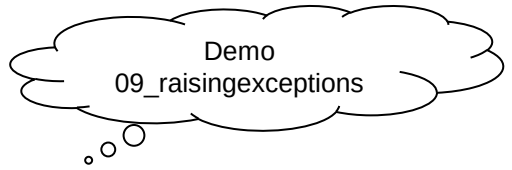
```
try:
    raise NameError('HiThere')
```

In order to catch an exception :

- `except` should refer to same exception thrown as class object

- **To capture above exception, write except clause as follows :**

```
try:
   Business Logic here...
except NameError:
    print('An exception flew by!')
    # raise
else:
   Rest of the code here...
```



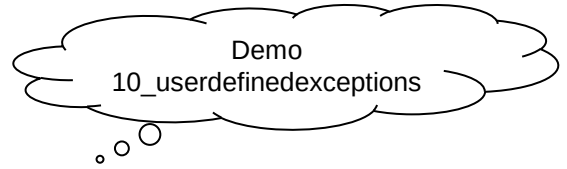
User Defined Exceptions

- **Created by deriving classes from standard built-in exceptions :**
  - Exception class may created that is subclassed from `RuntimeError`
  - Useful when more specific information is needed when an exception is caught
- **In `try:` block :**
  - User-defined exception is raised and caught in the `except` block
  - The variable e is used to create an instance of the class `Networkerror`

```
class Networkerror(RuntimeError):
   def __init__(self, arg):
      self.args = arg
```

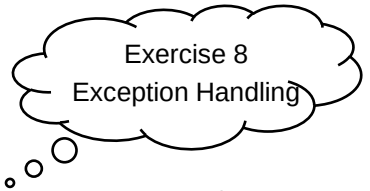
- **With above class exception is raised as follows :**

```
try:
   raise Networkerror("Bad hostname")
except Networkerror as e:
   print(e.args)
```



Summary : Exception Handling

- **Exceptions are events disrupting normal program flow :**
  - Python provides exception handling mechanism for unexpected errors
- **Code that might raise an exception is placed in a `try` block :**
  - This code is tested for exceptions
- **`try` block is followed by an `except` block :**
  - Control jumps to this block in case of exceptions and exceptions are handled there
- **`try` block may also be followed by a `finally` block :**
  - Is always executed whether or not the try-block raised an exception
- **Exceptions can have an argument :**
  - Value that gives additional information about the problem
- **Python has a number of predefined exceptions like :**
  - `IOError, ArithmeticError, KeyError, IndentationError`
- **Exceptions can be raised with the `raise` statement**
- **It is possible to create User Defined Exceptions :**
  - By deriving classes from standard built-in exceptions



© copyright : spiraltrain@gmail.com

SpiralTrain
developer courses

Course Schedule

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- Comprehensions
- Exception Handling
- **Python IO**
- Python Database Access
- Python Classes
- Optional : Python Libraries

- **Input and Output**
- **IO Module**
- **Opening Files**
- **File Open Modes**
- **Result of Calling open**
- **File Object Attributes**
- **Reading Binary Files**
- **Writing Binary Files**
- **Reading Text Files**
- **Writing Text Files**
- **File Positions**
- **Renaming and Deleting Files**
- **Directory Methods**
- **Creating Directories**

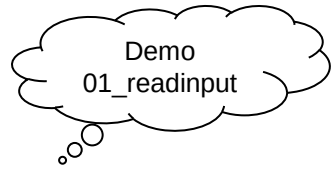
© copyright : spiraltrain@gmail.com

Input and Output

- **Simplest way to produce output is `print` function :**
  - Allows you to pass expressions, separated by commas, enclosed in parentheses

```
print ("Python is really a great language,", "isn't it? ")
```

- **Major difference between version 2 and 3 of the language :**
  - In version 3 print is a function
- **Standard input default comes from keyboard with input function**
- **`input([prompt])` function :**
  - Reads line from standard input, converts it to a string and strips trailing newline
- **Python also has File Object methods :**
  - Most of the file manipulation is done using a file object
- **Python also has OS Object Methods :**
  - Methods to process files as well as directories



IO Module

- **Core of the I/O system is implemented in io library module**
- **Consists of a collection of different I/O classes :**
  - `FileIO`
  - `BufferedReader`
  - `BufferedWriter`
  - `BufferedRWPair`
  - `BufferedRandom`
  - `TextIOWrapper`
  - `BytesIO`
  - `StringIO`
- **Each class implements a different kind of I/O**
- **Classes get layered to add features**

Opening Files

- `open` **function :**
  - Opens a file to be able to read or write a file later
  - Creates file object which can be used to call other support methods

```
file_object = open(file ,mode=r, buffering=-1, Encoding=None )
```

- `file` :
  - String value that contains the name of the file to be accessed
- `mode` :
  - Determines mode in which the file has to be opened, read, write, append etc.
  - `t` for text mode is default and optional, `b` is for byte mode
- `buffering` :
  - If buffering value is set to 0, no buffering will take place
  - If buffering value is 1, line buffering will be performed while accessing file
  - If buffering > 1 : Buffering performed with indicated buffer size

File Open Modes

- **Python distinguishes between binary and text I/O :**
  - In binary mode content is returned as bytes objects without any decoding
  - In text mode as `str`, bytes decoded using given or platform-specific encoding

Modes	Description
`r`	Opens file for reading only. File pointer is placed at beginning of the file. Default mode
`rb`	Opens file for reading only in binary format. File pointer is placed at beginning of file. Default mode
`r+`	Opens file for both reading and writing. File pointer will be at the beginning of the file
`rb+`	Opens file for both reading and writing in binary format. File pointer will be at the beginning of the file
`w`	Opens file for writing only. Overwrites file if file exists. If file does not exists, creates a new file for writing
`wb`	Opens file for writing only in binary format. Overwrites file if file exists. If file does not exist, creates new file for reading and writing
`w+`	Opens file for writing and reading. Overwrites existing file if file exists. If file does not exists, creates new file for reading and writing
`wb+`	Opens file for both writing and reading in binary format. Overwrites existing file if file exists. If the file does not exist, creates a new file for reading and writing
`a`	Opens a file for appending. File pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
`ab`	Opens a file for appending in binary format. File pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates new file for writing

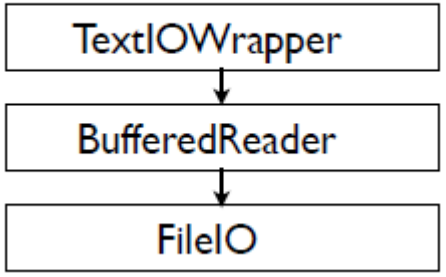
Result of Calling open

- **Varies depending on file mode and buffering**
- **Carefully study output of this for an existing file "`test.txt`" :**

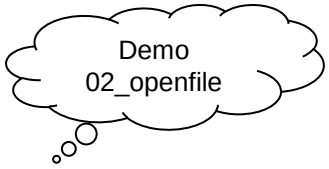
```
Python 3.2 (r32:88445, Feb 20 2011, 21:29:02) [MSC v.1500 32 bit (Intel)] on win
32
Type "help", "copyright", "credits" or "license" for more information.
>>> open("test.txt","rt")
<_io.TextIOWrapper name='test.txt' mode='rt' encoding='cp1252'>
>>> open("test.txt","rb")
<_io.BufferedReader name='test.txt'>
>>> open("test.txt","rb", buffering = 0)
<_io.FileIO name='test.txt' mode='rb'>
>>>
```

- **Result of opening a "text" file :**

```
open("foo.txt","rt")
```



- **This is very different from Python 2**



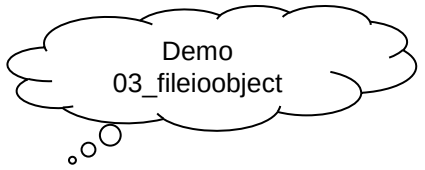
File Object Attributes

- **FileIO object represents raw unbuffered binary I/O**
- **File object has attributes with information about it**
- `file.closed` **: Returns true if file is closed, false otherwise**
- `file.mode` **: Returns access mode with which file was opened**
- `file.name` **: Returns name of the file**
- **Example :**

```
fo = open("test.txt", "wb")
print("Name of the file: ", fo.name)
print("Closed or not : ", fo.closed)
print("Opening mode : ", fo.mode)
```

- **Result :**

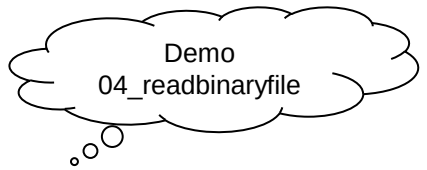
```
Name of the file:  test.txt
Closed or not :  False
Opening mode :  wb
```



Reading Binary Files

- **Opening a file in binary mode is simple but subtle :**
  - Reading bytes not strings, so there's no conversion for Python to do
- **Difference from opening it in text mode :**
  - b mode parameter
  - No encoding attribute
- `read` **method reads whole file or a number of bytes :**
  - `tell` moves file pointer to the beginning of the file
  - `seek` moves file pointer to a certain position in the file

```
fr = open('trilobyte.jpg', mode='rb')
fr.tell()
data = fr.read(3)       => read three bytes
print(type(data))      => type is bytes
fr.tell()
fr.seek(0)
data = fr.read()
print(data)
```



Writing Binary Files

- **Open binary files for writing without encoding attribute :**

```
fw = open('binaryfile04.txt', mode='wb')
```

- **Use `pack` method of `struct` module to prepare buffer for writing :**
  - Format string determines how packing takes place

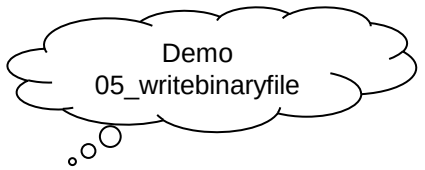
```
nr1 = 1; nr2 = 2; result = nr1 + nr2
buffer = pack("iii", nr1, nr2, result)
fw.write(buffer)
```

- **Data must be read like it is written using same format string :**

```
fr = open('binaryfile04.txt', mode='rb')
buffer = fr.read()
data = unpack("iii", buffer)
```

- **Returned data is a tuple that can be accessed as follows :**

```
num1 = data[0], num2 = data[1], res = data[2]
```



Writing Text Files

- **Open file for writing specifying mode and encoding :**

```
fw = open('textfilepolish.txt', mode='w', encoding='UTF-8')
```

- `write` **method writes any string to an open file :**
  - Parameter is the content to be written into the opened file
  - Does not add a newline character ('\n') to the end of the string

```
s = 'Polish text: ąćęłńóśźżĄĆĘŁŃÓŚŹŻ'
fw.write(s)
```

- **Always close the file when you are done :**

```
fw.close()
```

- **File** `textfilepolish.txt` **with following content is created :**

```
Polish text: Ä…Ä‡Ä™Å‚Å„Ã³Å›Å°Å¼Ä„Ä†ÄŘ�ÅƒÃ“ÅšÅ¹Å»
```

Reading Text Files

- **Open file for reading specifying mode and encoding :**

```
fr = open('textfilepolish.txt', mode='r', encoding='UTF-8')
```

- `read` **method reads a string from an open file :**
  - Parameter is the number of bytes to be read from the opened file

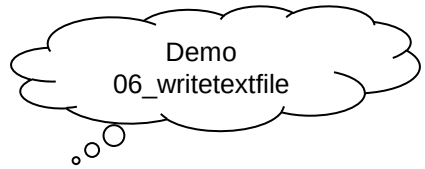
```
fileObject.read([count]);
```

- `read()` **starts reading from the beginning of the file :**
  - If count is missing then it tries to read as much as possible, may be until end of file
- **Example of reading and closing file** `textfilepolish.txt` **:**

```
data = fr.read()
print (data)
fr.close()
```

- **Result :**

```
Polish text: ąćęłńóśźżĄĆĘŁŃÓŚŹŻ
```



Closing Files

- `close` **method flushes unwritten data and closes file object :**
  - No more writing can be done after calling `close`
  - File is closed automatically when reference file object is reassigned to another file
- **Syntax :**

```
fileObject.close();
```

- **Example :**

```
fo = open("foo.txt", "wb")
print ("Name of the file: ", fo.name)
fo.close()
```

- **Result :**

```
Name of the file:  foo.txt
```

- **Good practice to use** `close()` **method to close a file**

File Positions

- `tell` **method retrieves current position within the file :**
  - Next read or write will occur at that many bytes from the beginning of the file
- `seek(offset[, from])` **method changes the current file position :**
  - Offset argument indicates the number of bytes to be moved
  - From argument specifies reference position from where bytes are to be moved
- **If from is set to 0 :**
  - Use beginning of the file as the reference position
- **If from is set to 1 :**
  - Use current position as the reference position
- **If from is set to 2 :**
  - Use end of the file as the reference position

Example File Positioning

- **Consider file `foo.txt` that was created before :**

```
fo = open("foo.txt", "r+")
str = fo.read(10);
print ("Read String is : ", str)
# Check current position
position = fo.tell();
print ("Current file position : ", position)
# Reposition pointer at the beginning once again
position = fo.seek(0, 0);
str = fo.read(10);
print ("Again read String is : ", str)
fo.close()
```

- **Result :**

```
Read String is :  Python is
Current file position :  10
Again read String is :  Python is
```

Renaming and Deleting Files

- **Files can be renamed and deleted :**
  - `os` module has methods for file-processing operations like renaming and deleting
  - `os` module must first be imported before methods can be called
- `rename` **method takes two arguments :**
  - Current filename and new filename

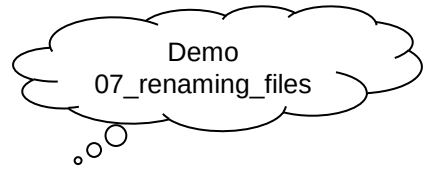
```
os.rename(current_file_name, new_file_name)
```

- **Rename an existing file from** `test1.txt` **to** `test2.txt` **:**

```
import os
os.rename( "test1.txt", "test2.txt" )
```

- **Delete an existing file** `test2.txt` **:**

```
import os
os.remove("text2.txt")
```



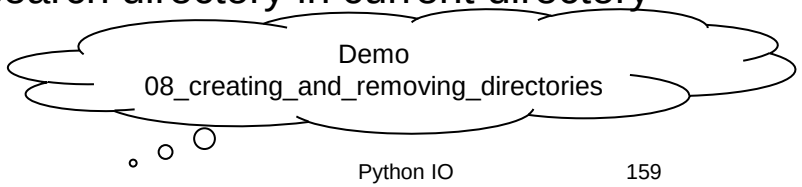
Creating and Removing Directories

- **Files are in directories and Python can handle directories too :**
  - os module has methods to create, remove, and change directories
- `mkdir()` **method creates directory in the current directory :**
  - Argument is name of the directory to be created
- **Syntax :** `os.mkdir("newdir")`
- **Example to create a directory test in current directory :**

```
import os
os.mkdir("test")
```

- `rmdir()` **method deletes the directory passed as an argument :**
  - Before removing a directory, all the contents in it should be removed
- **Syntax :** `os.rmdir('dirname')`
- **Example to remove** `"/tmp/test"` **directory :**
  - Give fully qualified name of directory otherwise search directory in current directory

```
import os
os.rmdir("/tmp/test")
```



Directory Methods

- `chdir()` **changes the current directory :**
  - Argument is the name of the directory that you want to make the current directory
- **Syntax :**

```
os.chdir("newdir")
```

- **Example changing a directory to `"/home/newdir"` :**

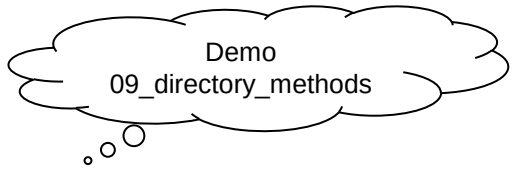
```
import os
os.chdir("/home/newdir")
```

- `getcwd()` **displays the current working directory**
- **Syntax :**

```
os.getcwd()
```

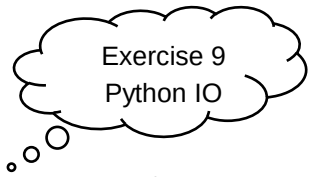
- **Example to give location of the current directory :**

```
import os
os.getcwd()
```



Summary : Python IO

- **Simplest way to produce output is to use the `print` statement :**
  - Allows you to pass zero or more expressions, separated by commas
- **Two built-in functions read a line of text from standard input :**
  - `raw_input` reads one line from standard input and returns it as string
  - `input` assumes input is a valid Python expression and returns evaluated result
- **Python also has File Object methods to manipulate files :**
  - Creating, opening files, closing files, reading, writing, renaming and deleting files
- **Files can be opened in several modes :**
  - For reading, for writing, for appending or combinations thereof
- **File Positioning :**
  - Methods can retrieve and change the current file position
- **Python also has OS Object Methods to manipulate directories :**
  - Creating and deleting directories
  - Changing and displaying the current directory



© copyright : spiraltrain@gmail.com

Course Schedule

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- **Python Database Access**
- Python Classes
- Optional : Python Libraries

- **Python DB API**
- **DB API Concepts**
- **Check Database Version**
- **Using with**
- **Inserting Data**
- **Prepared Statements**
- **Last inserted row id**
- **Retrieving Data**
- **Fetching Rows**
- **Dictionary Cursor**
- **Parameterized Queries**
- **Metadata**
- **Transactions**

© copyright : spiraltrain@gmail.com

Python DB API

- **Provides a standard interface from Python to databases :**
  - Most Python database interfaces adhere to this standard
  - Each database has its own module highly consistent with others
- **Influenced by ODBC, JDBC and Perl's DBI :**
  - Modules available for most of the popular relational databases
- **Standard is easy to implement and understand and this means :**
  - More and higher quality database third party modules
  - More supported backends
- **Python DB API 2.0 compatible modules :**
  - Sqlite3 part of the standard library
  - MySQLdb (MySQL)
  - psycopg2 (PostgreSQL)
  - cx_Oracle (Oracle)
  - mxODBC (SQL Server, DB2, Sybase, Oracle, etc.)
  - mxODBC Connect (same as mxODBC, but more platforms)

DB API Concepts

- **DB API uses two main concepts for processing queries :**
  - Connections and cursors
- **Connection objects logically wrap a database connection :**
  - Provide network/RPC access to the database
  - Provide means to handle database transactions
  - You can't execute statements on connection objects
- **Example :**

```
db_connection = sqlite.connect('sample.db')
```

- **Cursor objects used to manipulate and query the database :**
  - Created on connections using the `conn.cursor()` method
  - Cursor objects provide ways to execute statements and access results
- **Example :**

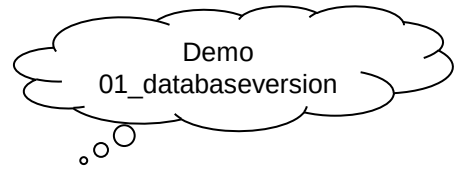
```
cursor = conn.cursor()
cursor.execute('create table testtable(id int, name varchar(254))')
```

Check Database Version

```
import sqlite3
import sys

con = None

try :
    con = sqlite3.connect('test.db')
    cur = con.cursor()
    cur.execute('SELECT SQLITE_VERSION()')
    data = cur.fetchone()
    print ("SQLite version: %s" % data)
except sqlite3.Error as e :
    print ("Error %s:" % e.args[0])
    sys.exit(1)
finally :
    if con:
        con.close()
```



Using with

- **By using the `with` statement the code is more compact :**
  - Using `with` the Python interpreter automatically releases the resources
  - This also provides error handling

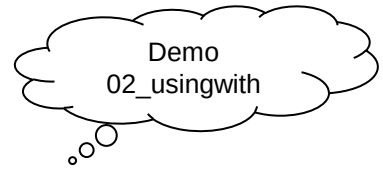
```
import sqlite3

con = sqlite3.connect('test.db')

with con:
    cur = con.cursor()
    cur.execute('SELECT SQLITE_VERSION()')

    data = cur.fetchone()
    print ("SQLite version: %s" % data)
```

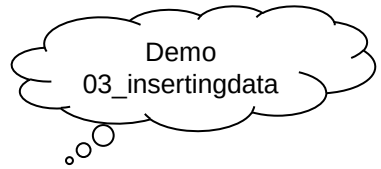
- **Using the `with` statement :**
  - Changes are automatically committed
  - Otherwise, we would have to commit them manually



Inserting Data

- **Cursor objects provides a `cursor.execute()` method :**
  - Execute SQL statements on a connection
- **Method also accepts :**
  - Sequence of parameters to customize the SQL statement

```
import sqlite3
con = sqlite3.connect('cars.db')
with con :
  cur = con.cursor()
  cur.execute("CREATE TABLE Cars(Id INT, Name TEXT, Price INT)")
  cur.execute("INSERT INTO Cars VALUES(1,'Audi',52642)")
  cur.execute("INSERT INTO Cars VALUES(2,'Mercedes',57127)")
  cur.execute("INSERT INTO Cars VALUES(3,'Skoda',9000)")
  cur.execute("INSERT INTO Cars VALUES(4,'Volvo',29000)")
  cur.execute("INSERT INTO Cars VALUES(5,'Bentley',350000)")
  cur.execute("INSERT INTO Cars VALUES(6,'Citroen',21000)")
  cur.execute("INSERT INTO Cars VALUES(7,'Hummer',41400)")
  cur.execute("INSERT INTO Cars VALUES(8,'Volkswagen',21600)")
```



Prepared Statements

- **If same query must be repeated multiple times :**
  - Can use parameterized queries with question marks or named parameters
  - Good for both performance and for security
- **Can use `executemany()` method for prepared statements :**

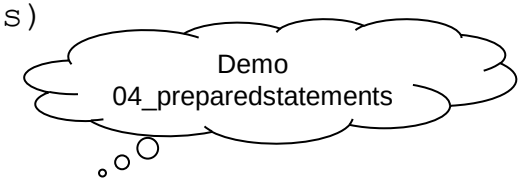
```
import sqlite3

cars = ((1, 'Audi', 52642), (2, 'Mercedes', 57127), (3, 'Skoda', 9000),
        (4, 'Volvo', 29000), (5, 'Bentley', 350000), (6, 'Hummer', 41400),
        (7, 'Volkswagen', 21600))

con = sqlite3.connect('test.db')

with con:
    cur = con.cursor()

    cur.execute("DROP TABLE IF EXISTS Cars")
    cur.execute("CREATE TABLE Cars(Id INT, Name TEXT, Price INT)")
    cur.executemany("INSERT INTO Cars VALUES(?, ?, ?)", cars)
```



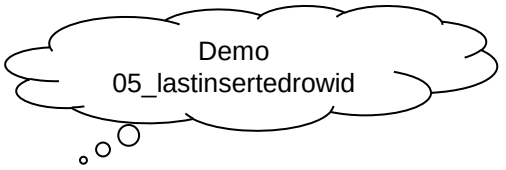
Last inserted row id

- **Sometimes we need to determine the id of last inserted row :**
  - In Python SQLite we use the `lastrowid` attribute of the cursor object

```
import sqlite3
con = sqlite3.connect(':memory:')

with con:
  cur = con.cursor()
  cur.execute("CREATE TABLE Friends(Id INTEGER PRIMARY KEY, Name TEXT);")
  cur.execute("INSERT INTO Friends(Name) VALUES ('Tom');")
  cur.execute("INSERT INTO Friends(Name) VALUES ('Rebecca');")
  cur.execute("INSERT INTO Friends(Name) VALUES ('Jim');")
  cur.execute("INSERT INTO Friends(Name) VALUES ('Robert');")

  lid = cur.lastrowid
  print ("The last Id of the inserted row is %d" % lid)
```



Retrieving Data

- **Data retrieval is done with fetch functions like `fetchall()` :**

```
import sqlite3

con = sqlite3.connect('cars.db')

with con:
    cur = con.cursor()
    cur.execute("SELECT * FROM Cars")

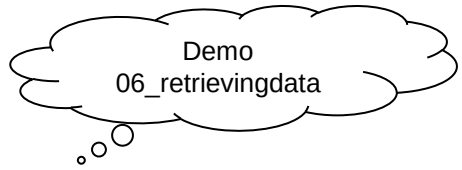
    rows = cur.fetchall()

    for row in rows:
        print (row)
```

```
(1, 'Audi', 52643)
(2, 'Mercedes', 57127)
(3, 'Skoda', 9000)
(4, 'Volvo', 29000)
(5, 'Bentley', 350000)
(6, 'Hummer', 41400)
(7, 'Volkswagen', 21600)
```

- `fetchall()` **gets all records and returns a result set :**
  - Actually the result set is a list of tuples
- **Other `fetch*()` methods read data from result set :**

```
cursor.execute('select * from testtable')
first_row = cursor.fetchone()
next_10_rows = cursor.fetchmany(10)
all_remaining_rows = cursors.fetchall()
```



Fetching Rows

- **Returning all data at a time may not be feasible :**
  - It is also possible to fetch rows one by one with `fetchone()` **:**

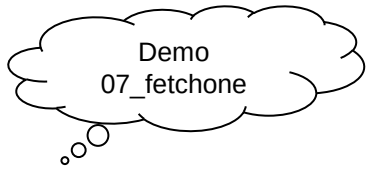
```
import sqlite3
con = lite.connect('test.db')

with con:
    cur = con.cursor()
    cur.execute("SELECT * FROM Cars")

    while True:
        row = cur.fetchone()

        if row == None:
            break
        print (row[0], row[1], row[2])
```

- `fetchone()` **method returns next row from the table :**
  - If there is no more data left, it returns `None`



Dictionary Cursor

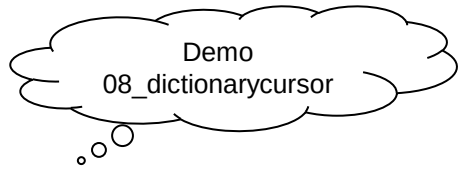
- **Default cursor returns data in a list of tuples :**
  - When we use a dictionary cursor, the data is sent in the form of Python dictionaries
  - This way we can refer to the data by their column names

```
import sqlite3
con = sqlite3.connect('cars.db')

with con:
    con.row_factory = sqlite3.Row

    cur = con.cursor()
    cur.execute("SELECT * FROM Cars")

    rows = cur.fetchall()
    for row in rows:
        print ("%s %s %s" % (row["Id"], row["Name"], row["Price"]))
```



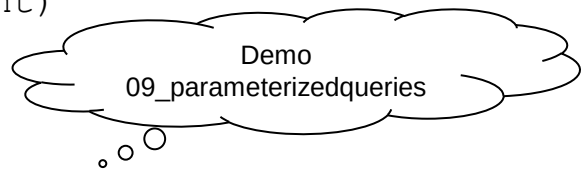
Parameterized Queries

- **Passing data from Python to the database :**
  - First discouraged option is quoting values and sending plain SQL to the database
  - Second option preferred using binding parameters in the SQL with place holders
- **Python sqlite3 module supports two types of placeholders :**
  - Question marks and named placeholders

```
import sqlite3

uId = 1
uPrice = 62300

con = sqlite3.connect('test.db')
with con:
    cur = con.cursor()
    cur.execute("UPDATE Cars SET Price=? WHERE Id=?", (uPrice, uId))
    con.commit()
    print ("Number of rows updated: %d" % cur.rowcount)
```



Metadata

- **Metadata is information about the data in the database :**
  - Contains information about the tables and columns like name and datatypes

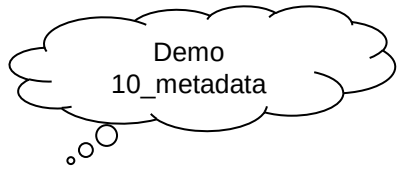
```
import sqlite3
con = sqlite3.connect('cars.db')

with con:
    cur = con.cursor()
    cur.execute('PRAGMA table_info(Cars)')

    data = cur.fetchall()
    for d in data:
        print (d[0], d[1], d[2])
```

```
0 Id INT
1 Name TEXT
2 Price INT
```

- **Metadata in SQLite can be obtained using `PRAGMA` command :**
  - SQLite objects may have attributes, which are metadata
  - Obtain specific metatada from querying the SQLite system sqlite_master table



Transactions

- **Transactions logically wrap statements into blocks of execution :**
  - Transactions are units of recovery
- **Main benefit is that you can undo changes very easily :**
  - Ensures data consistency at all times
  - Not all databases provide transactions !
- **Transactions conform to ACID principles :**
  - Atomicity, Consistency, Isolation, Durability
- **DB API supports transactions if the database supports them :**
  - Via methods on the connection object
- **Write all changes of the last transaction block to the database :**

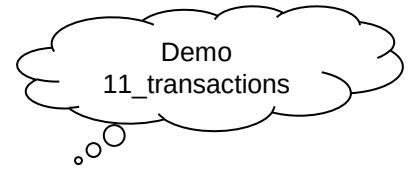
```
conn.commit()
```

- **Undo all changes applied to the database on the connection :**

```
conn.rollback()
```

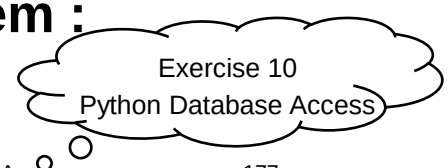
Committing Data

```
import sqlite3
import sys
try:# con = sqlite3.connect('friends.db', isolation_level=None) => autocommit
    con = sqlite3.connect('friends.db')
    cur = con.cursor()
    cur.execute("DROP TABLE IF EXISTS Friends")
    cur.execute("CREATE TABLE Friends(Id INTEGER PRIMARY KEY, Name TEXT)")
    cur.execute("INSERT INTO Friends(Name) VALUES ('Tom')")
    cur.execute("INSERT INTO Friends(Name) VALUES ('Rebecca')")
    cur.execute("INSERT INTO Friends(Name) VALUES ('Jim')")
    cur.execute("INSERT INTO Friends(Name) VALUES ('Robert')")
    #con.commit()       // data not written until commit is executed
except sqlite3.Error as e:
    if con:
        con.rollback()
    print ("Error %s:" % e.args[0])
    sys.exit(1)
finally:
    if con:
        con.close()
```



Summary : Python Database Access

- **Python DB API is standard interface from Python to databases :**
  - Each database has its own module highly consistent with others
- **DB API uses two main concepts for processing queries :**
  - Connections and cursors
- **Connection objects logically wrap a database connection :**
  - Provide means to handle database transactions
- **Cursor objects used to manipulate and query the database :**
  - Created on connections using the `conn.cursor()` method
- **If same query must be repeated multiple times :**
  - Can use parameterized queries with question marks or named parameters
- **Using `with` Python automatically releases the resources :**
  - Changes are automatically committed
- **DB API supports transactions if database supports them :**
  - Via methods on the connection object



© copyright : spiraltrain@gmail.com

SpiralTrain
developer courses

Course Schedule

- Python Intro
- Variables and Data Types
- Data Structures
- Control Flow
- Functions
- Modules
- Comprehensions
- Exception Handling
- Python IO
- Python Database Access
- **Python Classes**
- Optional : Python Libraries

- **Object Orientation in Python**
- **Creating Classes**
- **Class Members**
- **Creating and Using Objects**
- **Accessing Attributes**
- **Property Syntax**
- **Built-in Class Attributes**
- **Constructors and Destructors**
- **Destroying Objects**
- **Static Methods**
- **Data Hiding**
- **Encapsulation**
- **Overriding Methods**

© copyright : spiraltrain@gmail.com

Object Orientation in Python

- **Python is object-oriented language :**
  - Creating and using classes and objects is built in the language
  - Classes are created in `Python` with the `class` keyword
- **Class :**
  - Blueprints for a set of related objects with both description and implementation
  - Defines a set of attributes and methods that characterize any object of the class
  - Attributes are data members, Methods, accessed via dot notation
- **Object or Instance :**
  - An individual object of a certain class which has own identity
  - A unique instance of a data structure that's defined by its class
  - State represents current values of attributes
  - State differs per object, behavior is shared by all instances
- **Inheritance :**
  - Subclass inherits attributes and methods from a superclass
  - New attributes and methods may be added

Example Class and Objects

Car Class		Object Car A	Object Car B
Instance Variables	Plate Number	ABC 111	XYX 123
	Color	Blue	Red
	Manufacturer	Mitsubishi	Toyota
	Current Speed	50 km/h	100 km/h
Instance Methods	Accelerate Method		
	Turn Method		
	Brake Method		

OO Terminology Overview

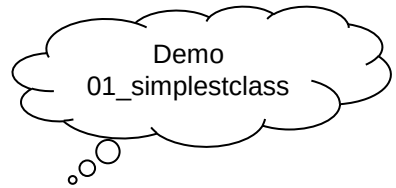
- **Instance variable :**
  - Variable defined inside a method and belongs only to current instance of a class.
  - Also called data member for instance fields
- **Class variable :**
  - Variable that is shared by all instances of a class
  - Also called data member for class fields
  - Class variables are defined within a class but outside any of the class's methods
  - Class variables aren't used as frequently as instance variables are
- **Function overloading :**
  - The assignment of more than one behavior to a particular function
  - The operation performed varies by the types of objects (arguments) involved.
- **Operator overloading :**
  - The assignment of more than one function to a particular operator

Creating Classes

- `class` **statement creates a new class definition :**
  - Name of the class immediately follows the keyword `class` followed by a colon :

```
class ClassName:
   'Optional class documentation string'
   class_suite
```

- **The class has a documentation string :**
  - Can be access via `ClassName.__doc__`
- `class_suite` **consists of all the component statements :**
  - Class members
  - Data attributes
  - Functions

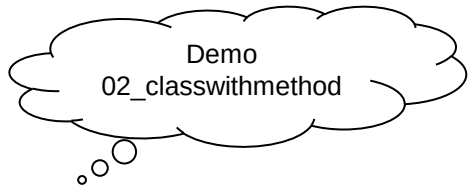


Example Creating Classes and Objects

```
class Employee :
   'Common base class for all employees'
   empCount = 0
   def __init__(self, name, salary):
      self.name = name
      self.salary = salary
      Employee.empCount += 1
   def displayCount(self):
     print ("Total Employee %d" % Employee.empCount)
   def displayEmployee(self):
      print ("Name : ", self.name,  ", Salary: ", self.salary)
```

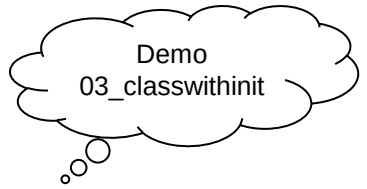
- **To create instances of a class :**
  - Call class using class name and pass arguments that `__init__` method accepts
- **Create object of `Employee` class :**

```
emp1 = Employee("Albert", 2000)
```



Class Members

- `empCount` :
  - Class variable whose value would be shared among all instances of a this class
  - Accessible as `Employee.empCount` from inside the class or outside the class
- `__init__()` :
  - Special method which is called class constructor or initialization method
  - Called by Python when you create a new instance of this class
- **Instance methods :**
  - Declared like normal functions but first argument to each method is `self`
  - No need to include this argument when you call methods since Python adds it
- `self` **must be explicitly listed as first argument to method :**
  - Instance variables are referred to with "`self.XXX`"
- `self` **is a reference to the instance :**
  - Reference to the container for state of object
  - In many OO languages, instance is hidden in method definitions
  - In Python, instance is visible and *explicit* in method definitions



Creating and Using Objects

- **Creating objects :**

```
"This would create first object of Employee class"
emp1 = Employee("Albert", 2000)
"This would create second object of Employee class"
emp2 = Employee("Werner", 5000)
```

- **Access object's attributes using dot operator with object :**

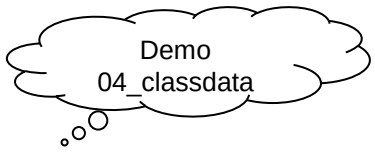
```
emp1.displayEmployee()
emp2.displayEmployee()
```

- **Access class variable using class name :**

```
print("Total Employee %d" % Employee.empCount)
```

- **Result :**

```
Name :  Albert ,Salary:  2000
Name :  Werner ,Salary:  5000
Total Employee 2
```



Accessing Attributes

- **In Python can add, remove, or modify attributes at any time :**

```
emp1.age = 7  # Add an 'age' attribute
emp1.age = 8  # Modify 'age' attribute
del emp1.age  # Delete 'age' attribute
```

- **There are alternative functions to access attributes :**
  - `getattr(obj, name[, default])` : Access the attribute of object
  - `hasattr(obj,name)` : Check if attribute exists or not
  - `setattr(obj,name,value)` : Set or create attribute
  - `delattr(obj, name)` : Delete an attribute
- **Example code :**

```
hasattr(emp1, 'age')    # Returns true if 'age' attribute exists
getattr(emp1, 'age')    # Returns value of 'age' attribute
setattr(emp1, 'age', 8) # Set attribute 'age' at 8
delattr(empl, 'age')    # Delete attribute 'age'
```

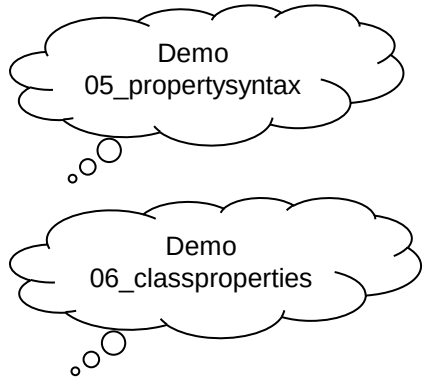
Property Syntax

- **Attributes of class can be accessed using get and set functions :**
  - Allow set function to have side effect such as a value check

```
class SquareGetterSetter(object):
    def get_length(self):           # Java-style
        return self._length
    def set_length(self, length):   # Java-style
        self._length = length
```

- **However Python also supports property syntax :**
  - Setters and getters are hidden by propery syntax and this is more pythonic way
  - May be added after class is used by clients without breaking them

```
class SquarePropertySyntax(object):
    def get_length(self):
        return self._length
    def set_length(self, length):
        self._length = length
    length = property(get_length, set_length)
```



Built-In Class Attributes

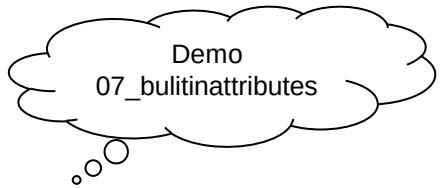
- **Every Python class has following built-in attributes :**
  - `__dict__` : Dictionary containing the class's namespace
  - `__doc__` : Class documentation string, or None if undefined
  - `__name__` : Class name.
  - `__module__` : Module name in which the class is defined
    This attribute is "__main__" in interactive mode.
  - `__bases__` : A possibly empty tuple containing the base classes
    In the order of their occurrence in the base class list
- **Can be accessed using dot operator like any other attribute**

Example Accessing Built-in Class Attributes

```
print ("Employee.__doc__:", Employee.__doc__)
print ("Employee.__name__:", Employee.__name__)
print ("Employee.__module__:", Employee.__module__)
print ("Employee.__bases__:", Employee.__bases__)
print ("Employee.__dict__:", Employee.__dict__)
```

- **Result :**

```
Employee.__doc__: Common base class for all employees
Employee.__name__: Employee
Employee.__module__: __main__
Employee.__bases__: (<class 'object'>,)
Employee.__dict__: {'__module__': '__main__', 'displayCount':
<function displayCount at 0xb7c84994>, 'empCount': 2,
'displayEmployee': <function displayEmployee at 0xb7c8441c>,
'__doc__': 'Common base class for all employees',
'__init__': <function __init__ at 0xb7c846bc>}
```



Destroying Objects

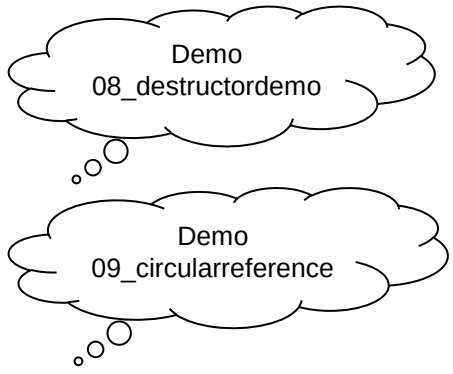
- **Python deletes unneeded objects automatically to free space :**
  - This process of periodically reclaims of memory is termed Garbage Collection
- **Python's garbage collector runs during program execution :**
  - Triggered when an object's reference count reaches zero
- **Reference count changes as nr of aliases pointing to it change :**
  - Increase when it's assigned a new name
  - Increase when placed in a container (list, tuple, or dictionary)
  - Decrease when deleted with `del`
  - Decrease when reference is reassigned or its reference goes out of scope
- **Reference count reaches zero :**
  - Python collects it automatically
- `__del__()` **destructor :**
  - May print the class name of an instance that is about to be destroyed
  - Might be used to clean up any non-memory resources used by an instance

Example Destructor

```
class Point:
   def __init( self, x=0, y=0):
      self.x = x
      self.y = y
   def __del__(self):
      class_name = self.__class__.__name__
      print (class_name, "destroyed")
pt1 = Point()
pt2 = pt1
pt3 = pt1
print (id(pt1), id(pt2), id(pt3)) # prints id's of objects
del pt1
del pt2
del pt3
```

- **Result :**

```
3083401324 3083401324 3083401324
Point destroyed
```



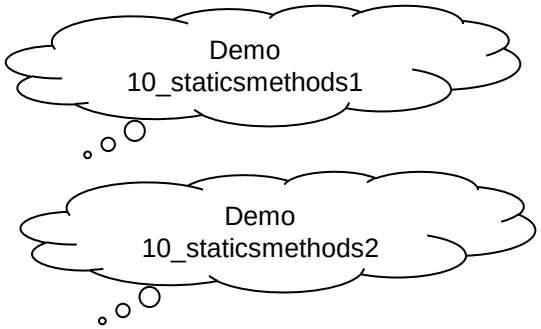
Static Methods

- **Static methods do not receive `self` as first parameter :**
  - May access data associated with the class instead of instance
- **Method can be made `static` in two ways :**
  - Annotate method with `@staticmethod` decoration
  - Exporting by calling `staticmethod` with method as parameter and return value

```
class Account :
    interestRate = 10
def getInterestRate1():
        return Account.interestRate
    getInterestRate1 = staticmethod(getInterestRate1)
    @staticmethod
    def getInterestRate2():
        return Account.interestRate

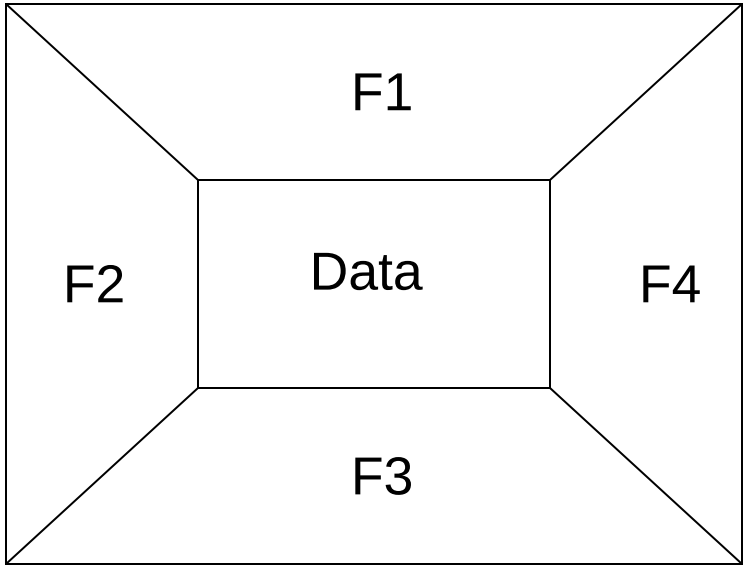
print (Account.getInterestRate1())
print (Account.getInterestRate2())
```

- `static` **method typically is called through the class**



Encapsulation

- **Well known object oriented concept**
- **Internal class data hidden by functions**
- **Result is implementation hiding and modularity**



Data Hiding

- **Many object oriented languages support data hiding :**
  - Attributes and methods may have modifiers like public, private and protected
- **Python does not support real data hiding as other languages :**
  - Attributes and methods can however be made more difficult to access
- **Attributes made private with double underscore prefix in name :**

```
__secretCount = 0
```

- **Attributes will not be directly visible to outsiders :**
  - Access of such attributes from outside should include class name

```
print(s._Secretive__secretCount)
```

- **Methods can also be protected with double underscore as prefix :**

```
def __inaccessible(self):
        print("Bet you can't see me...")
```

- **Methods can accessed from the outside in the following way :**

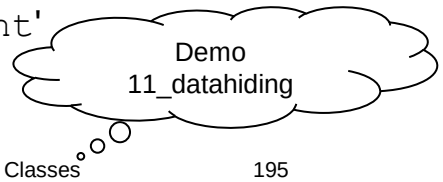
```
s._Secretive__inaccessible()
```

Example Data Hiding

```
class Secretive:
    __secretCount = 0
    def count(self):
        self.__secretCount += 1
        print (self.__secretCount)
s.count()
s.count()
print (s._Secretive__secretCount)
print (s.__secretCount)
```

- **Result :**

```
1
2
2
Traceback (most recent call last):
  File "", line 21, in <module>
    print (s.__secretCount)
AttributeError: 'Secretive' object has no attribute '__secretCount'
```



Override Base Methods

- **Generic functionality that you can override in your own classes :**
- `__init__ ( self [,args...] ) :`
  - Constructor (with any optional arguments)

    ```
    obj = className(args)
    ```
- `__del__( self ) :`
  - Destructor, deletes an object

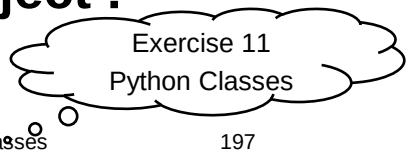
    ```
    dell obj
    ```
- `__repr__( self ) :`
  - Evaluatable string representation

    ```
    repr(obj)
    ```
- `__str__( self ) :`
  - Printable string representation

    ```
    str(obj)
    ```

Summary : Python Classes

- **Classes are blueprints for a set of related objects :**
  - Define both attributes and methods and are created with `class` keyword
- **Objects are instances of classes :**
  - Each object has its own state and identity, instance methods have `self` parameter
- **Constructor is method named `__init__()` :**
  - Creates object and called by assigning a variable of the object type
- **To get and set attributes Python supports property syntax :**
  - Setters and getters are hidden by propery syntax and this is more pythonic way
- **Python deletes unneeded objects automatically to free space :**
  - This process of periodically reclaims of memory is termed Garbage Collection
- **Encapsulation is hiding internal data through methods :**
  - `private` instance variables have double underscores `__var__`
  - Can be accessed through a scrambled name
- **Static methods and properties bound to class not to object :**
  - Static data only exists once per class rather than per instance



© copyright : spiraltrain@gmail.com